TWIN-64
Architecture Document

Helmut Fieres
December 17, 2025

Contents

List of Figures

List of Tables

Part I

Introduction

Introduction

Designers of CPUs in the seventies and early eighties almost all used a microcoded approach with hundreds of instructions. RISC was just beginning to emerge. Registers were typically not fully general-purpose but often had special functions or meanings, and many instructions were rather complicated, though designers believed them useful. Compiler writers, however, often used only a small subset, ignoring these complex instructions because they were too specialized. In the late eighties and nineties the shift moved to RISC-based designs, with large sets of registers, fixed instruction word lengths, large virtual memory address ranges, and instructions that were in general much more pipeline-friendly. Today, processors are highly complex, with superscalar deep pipelines and multilevel cache hierarchiesjust a few of the hallmarks of modern CPU design. CPU designs have become so large and complex that no single person can completely understand them. But what if there were a simple design that one person could fully understand?

Twin-64

Welcome to Twin-64. Twin-64 is a simple 64-bit CPU with a register-memory model and segmented virtual memory. The design is heavily influenced by Hewlett-Packards PA_RISC architecture, which was initially a 32-bit RISC-style load/store machine. Almost all of the key architectural features come from there. However, other processors such as the Data General MV8000, the DEC Alpha, the IBM PowerPCwere and more lately RISC-V also influential or at least investigated for their design choices. In addition, the Blitz-64 architecture was studied. Blitz-64, a design developed at Portland State University, offered an interesting approach in that it only supports 64-bit signed arithmetic, avoiding common errors from mixing signed and unsigned arithmetic.

Where does the name Twin-64 come from? Obviously, the 64 indicates that the CPU is a 64-bit CPU. The Twin part will become clearer when implementations of the CPU are described in later chapters. One implementation represents a dual-issue instruction design with two ALUs for computation. Nevertheless, a basic implementation could just offer single-issue, single-ALU execution as well.

Design Goals

While it is not a design goal to build an industry-strength, competitive CPU, the CPU should nevertheless be useful and largely follow established design practices. For example, instructions should not be invented just because they seem useful, but rather designed with the assembler and compilers in mind. Instruction set design is CPU design. Twin-64 instructions will be hardwired and are in general pipeline-friendly, avoiding data and control hazards and stalls where possible.

This book

This document serves as a comprehensive guide to the Twin-64 processor, its instruction set, and runtime environment. It is organized into several parts, each focusing on a key aspect of the system.

Part Two introduces the Twin-64 architecture, providing an overview of the overall system and processors structure, core components, and design philosophy. This sets the stage for understanding how instructions are executed and how data moves through the system.

Part Three presents the instruction set in detail. It covers computational, immediate, memory reference, branch, and system control instructions. These chapters serve as the definitive reference for programmers and designers alike, offering precise information on encoding, operation, and usage.

Part Four explores the I/O subsystem, detailing the mechanisms by which the processor communicates with external devices and handles input/output operations efficiently.

Part Five discusses the runtime environment and software conventions. It explains program interaction with Twin-64, including calling conventions, exception handling, and runtime support for higher-level languages.

Part Six presents a reference implementation of the architecture and instruction set processor in a simulator environment.

Part Seven examines processor design. While multiple design approaches are of course possible, this section provides a reference implementation that highlights key architectural concepts, practical trade-offs, and performance considerations.

Finally, the **Appendix** provides supplementary material, including additional design notes, tables, and clarifications to support the main text and offer deeper insight into the Twin-64 system.

Part II

Architecture

System Organization

This chapter introduces Twin-64 by highlighting the major components and design concepts.

A RegisterMemory Architecture

Modern RISC CPUs are typically out–of–order load / store designs and feature a large number of general-purpose registers. Operations solely take place between registers. Twin-64 follows a slightly different route. It has fewer general registers, and in addition to register-register computation, a register-memory model is also supported. In contrast to similar historical register-memory designs, Twin-64 has no operations that both read from and write to memory in the same instruction. For example, there is no increment memory instruction, since this would require two memory operations and is generally not pipeline-friendly.

Twin-64 offers only a few addressing modes for the operand, combined with a fixed instruction word length. For most instructions one operand is a register, while the other is a flexible operand that can be an immediate, another register, or a memory address from which data is obtained or stored. The result is placed in the register, which is also the first operand. These types of machines are also called two-address machines.

System

A Twin-64 system is organized around a central system bus that interconnects the key components of the architecture. Processors, main memory modules, and I/O modules are all attached as peers to this bus, enabling uniform communication and data exchange. Larger configurations may also include I/O adapters, which bridge slower peripheral buses to the central system bus while preserving a consistent programming model.

Processors initiate memory and I/O transactions via the bus and can also receive external events such as interrupts or system resets through the same interface. This modular structure provides flexibility in system design, allowing configurations to scale from simple processor-memory systems to complex multiprocessor environments with diverse I/O subsystems.

Processor

The Twin-64 processor implements the instruction set architecture and provides the computational resources of the system. It contains a general register set, control and status registers, and the program state register. To support virtual memory, each processor includes a Translation Lookaside Buffer (TLB) that translates virtual addresses into physical addresses. Instruction and data caches, while optional in principle, are essential for performance and tightly integrated

with the memory interface. Regardless of configuration, all processor-generated addresses are virtual and must be resolved by the TLB before memory access.

Virtual Memory

Twin-64 offers a large global address range, organized in regions. A region number and offset together form the global virtual address. regions are also associated with access rights and protection identifiers. The CPU itself can run in user or privileged mode. Twin-64 always generates virtual addresses at the CPU level. The global virtual memory range is a 52-bit space organized into a 20-bit region ID and a 32-bit byte offset. There are 2^{20} regions with 2^{32} bytes each.

Virtual addresses are translated to physical addresses by the TLB when memory is referenced. For memory management purposes, the virtual address range can also be viewed as a set of pages. A region therefore contains 2^{20} pages of 4 KB each. A virtual page number, combining the region and the page number within that region, is a 2^{40} value. Address translations are performed at the page level.

The first regions in the virtual address space are special in that they actually represent the physical address range. An address in that range bypasses the TLB and accesses physical memory directly.

A TLB can optionally support different page sizes in addition to the architected 4 KB page size. Supported sizes are 4 Kbyte, 64 Kbytes, 1 Mbyte and 16 MByte. It is very common for the operating system and data to be mapped in consecutive physical pages, which can then be covered by a larger page size.

Security and Protection Support

Twin-64 implements a protection model along two dimensions. The first dimension is **access type control** and privilege-level checking. Each page is associated with a page type. Page types include read-only, readwrite, execute, and gateway. There are two privilege levels: user mode and supervisor mode. Access type and privilege level together form the access control information, which is checked for each instruction and data memory access.

The second dimension of protection is the region ID itself. Twin-64 allows a set of region IDs to be recorded in processor control registers. For each access to a region that has the protection check bit set, one of the region ID control registers must match the region ID. If not, a protection violation trap is raised.

Privilege Changes

Instructions can only access data or execute instructions when the privilege level matches the required privilege level. Two or four levels are the most common implementations. Changing between levels is often done with a kind of trap operation to higher-privilege software, for

example the operating system kernel. However, traps are expensive, as they cause the CPU pipeline to be flushed when entering a trap handler.

Twin-64 implements gateways for privilege promotion. A special option on the unconditional branch instruction raises the privilege level when the instruction is executed on a gateway page, setting the privilege level to that of the gateway page. Returning from a higher privilege level to a code page with lower privilege level automatically resets the lower privilege level. A branch to a higher-privilege page that is not a gateway page results in a privilege violation trap.

Debugging Support

A fundamental requirement is the ability to set instruction and data breakpoints. An instruction breakpoint is triggered by encountering the `TRAP` instruction. The break instruction causes an instruction break trap and passes control to the trap handler. Since break instructions are normally not present in the instruction stream, they must be inserted dynamically in place of the instruction normally found at that address. Upon continuation, if the breakpoint is still valid, it must be reinserted after the original instruction is executed. To facilitate this mechanism, a bit in the status word provides an easy way to trap again after the instruction has been executed.

Data breakpoints are traps taken when a data reference to a page occurs. They do not modify the instruction stream. Depending on the data access, the trap may occur before or after the instruction that accesses the data. A write operation is trapped before the data is written, while a read instruction is trapped after the original instruction has executed.

Input/Output

Twin-64 implements a memory-mapped I/O architecture. A portion of the physical address space is reserved for I/O modules, which respond to memory load and store instructions directed to their assigned addresses. Standard page-level protection applies during address translation, ensuring that only privileged software (typically device drivers) can safely access I/O regions. User programs may be granted direct access to I/O by mapping a user-level I/O page into virtual memory, without compromising overall system integrity.

Direct I/O is performed using the standard load and store instructions to access I/O modules. These operations are entirely controlled by software and may be executed in user mode if the virtual mapping allows it.

Direct Memory Access (DMA) modules can transfer data between memory and I/O devices independently of the processor. DMA transfers operate on physical memory, requiring that the relevant pages be locked and protected from conflicting access by the operating system. DMA transactions are initiated by issuing DMA commands, and command chaining is supported to enable complex or continuous data transfers without processor intervention.

Interrupt System

Interrupts and exceptions are events that occur asynchronously to instruction execution. Depending on the type, they occur either during the execution of an instruction or between instructions. An example of the former is an arithmetic overflow trap; an example of the latter is an external interrupt. Depending on the interrupt or exception type, the instruction is either restarted or execution continues with the next instruction.

Twin-64 implements a simple interrupt system. Each processor features an interrupt input register and an interrupt mask register. When an I/O module receives a command, it also receives information about which interrupt bit to set and which target module ID to send the interrupt to upon completion of the request. The interrupt data is compared to the interrupt mask register. If the particular bit is enabled and interrupts are globally enabled, the target processor enters the interrupt handler.

Since raising an interrupt is simply a matter of storing the interrupt request into the processors interrupt register, I/O modules as well as processors themselves can raise interrupts on a target processor. Processors can also send interrupts to their peers, enabling low-level processor-to-processor communication.

Processing Resources

This chapter describes the processing resources and data types in a Twin-64 system. Twin-64 provides a set of general-purpose registers, a set of control registers, and the Program State Word (PSW). The general-purpose registers are used for computation and address storage, the control registers manage processor state, and the PSW holds both the current instruction address and processor status.

General Registers

Twin-64 features a set of 16 general purposes registers. They are used for all computations including address arithmetic.

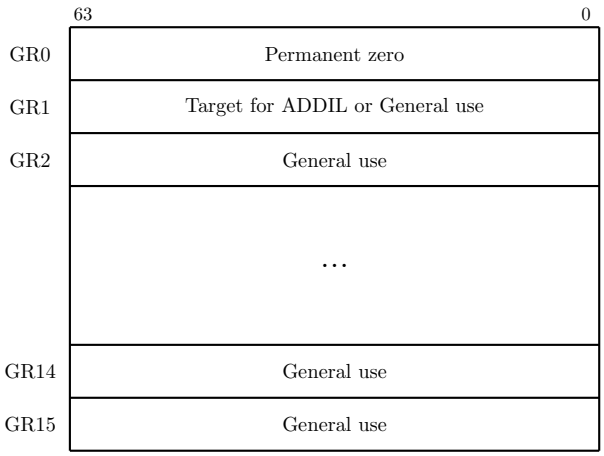


Figure 1: General registers

Some general registers have dedicated purposes. Register zero always returns a zero value when read, and writing to it has no effect. Although the CPU has only 16 general registers, implementing such a register greatly simplifies instruction design, for example by providing a target when the result is not needed. General register one is a scratch register and also serves as an implicit target for certain instructions. Other general registers may have dedicated roles as defined by the runtime architecture conventions.

Control Registers

Twin-64 has 16 defined control registers, numbered `CR0` to `CR15`. Two instructions allow moving data between a control register and a general register in either direction. A move to a control register always transfers the full 64-bit word. A move from a control register to a general register transfers the value right-aligned to the size of the control register. The following figure depicts the control registers defined for Twin-64.

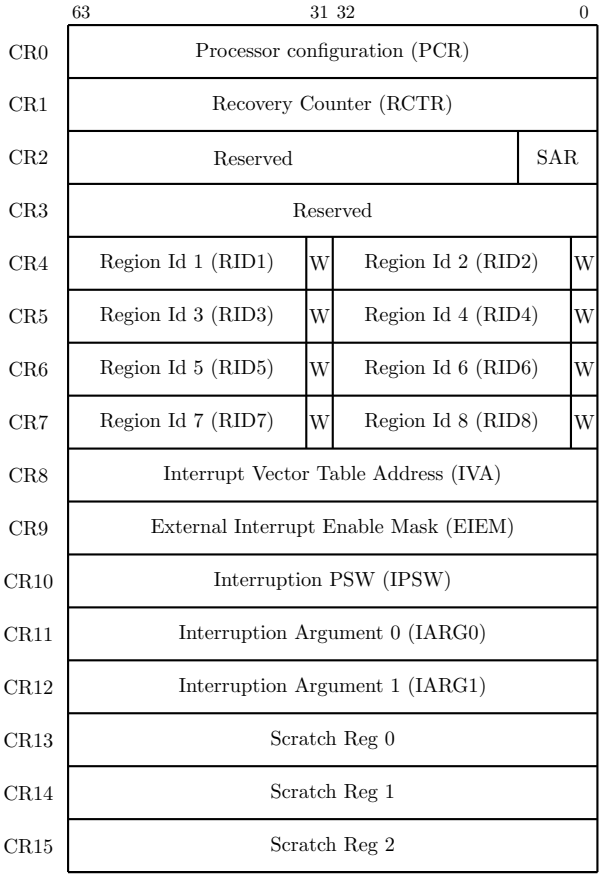


Figure 2: Control registers

Note: We may need more than 16 control registers. Under evaluation.

Processor Configuration Register

The processor configuration register contains information about the current system and processor hardware setup. Some of its fields are read-only and set directly by the hardware, while others can be modified by processor-dependent code routines.

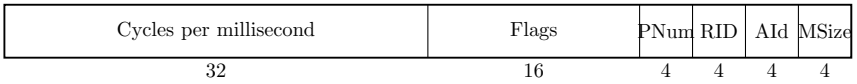


Table 1: Processor Configuration Register Fields

Field	Purpose
Memory Size (MSize)	The available physical memory in 4 Gbyte increments. The minimum size is 4 Gbyte, encoded as a zero. The maximum size is 64 Gbyte encoded as a 15.
Architecture Id (AId)	...

Continued on next page

Field	Purpose
Processor Id Id (RID)	...
Processor Core Num	in a multiprocessor configuration, the unique processor number is assigned during system startup.
Flags (tbd)	e.g default endian, etc.
Cycles per millisecond	the cycle per millisecond specifies the number of processor cycles for a millisecond.

Recovery Counter

When enabled, the recovery counter decrements on each instruction cycle. When the leftmost bit becomes a one and the R bit in the processor status word is set, a recovery trap is scheduled.

Shift Amount Register

The shift amount register (`SAR`) is a 6-bit register used by the extract, deposit and double shift instruction when a dynamic position is specified in the instruction. The valid number range is zero to 63.

Region Id Registers

Twin-64 allows to record a set of up to 8 region IDs in the processor control registers `CR4` to `CR7` which are accessible to the currently executing process. For each access to a region that has the protection check bit set, one of the protection Id control registers must match the region Id. If not, a protection violation trap is raised. The rightmost bit of each of the eight RIDs is the write disable (W) bit. When the bit is set, that RID data cannot be used to grant write access. See also the chapter on addressing and access control.

Interrupt Vector Table Address

The Interruption Vector Table Address (IVA) control register, `CR8`, holds the base address of an array of service routines assigned to the different interruption classes. The lower 12 bits of the IVA are reserved and must be set to zero, meaning any address written to it must be aligned to a page boundary. The IVA is always located in region zero. The array of interruption service routines is indexed by interruption numbers, as described in the chapter on interrupts.

External Interrupt Enable Mask

The External Interrupt Enable Mask (EIEM) is stored in `CR15)`. It is a 64-bit register, with each bit corresponding to one of the 64 external interrupts. A cleared bit in the EIEM masks the external interrupt associated with that position.

Interruption PSW

Upon an interrupt, the address of the interrupted instruction and the processor status bits are saved to this control register.

Interruption Argument Registers

The Interruption Argument Registers are used to pass an instruction and a virtual address to an interruption handler. The values of these registers for each interruption class are described in the chapter on interrupts.

Scratch Registers

Control registers `CR13`, `CR14`, and C`CR15` are scratch registers used by interrupt handlers or similar routines. `CR13` and `CR14` are privileged and can only be accessed from code running at a privileged level, whereas `CR15` is accessible from any privilege level.

Program State Register

The processor state register contains the virtual address of the current instruction along with the processor status flags.

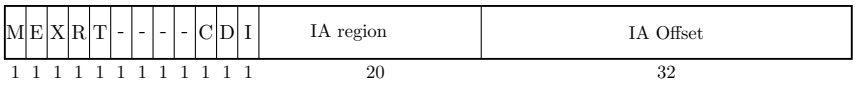


Figure 3: Program State Register

Note: Another option is to have a separate status word. Under evaluation.

Instruction Address

The instruction address portion of the PSW registers contains the region id and the byte offset into the region for the currently executing instruction.

Processor Status

Table 2: Status Field Bits

Bit	Purpose
M	High priority machine check. When set, high-priority machine checks are masked. This bit is set after an HPMC and cleared after all other interruptions.
E	Little endian access enable. When set, memory access is in little endian format, else big endian format. Endian support is an optional feature. If not implemented, all access is in big endian format.
R	Recovery counter enable. When set the recovery counter trap is issued when the recovery counter becomes negative.
X	When set, the processor runs in privileged (supervisor) mode.
T	When set, taken branch traps are enabled. Any branch taken will result in a trap.
I	When set, external interrupts are enabled.
V	Reserved.
C	Instruction memory break disable. The C-bit is cleared after the execution of each instruction, except for the `RFI` instruction which may set it. When set, instruction memory break traps are disabled. This bit allows a simple mechanism to trap on an instruction and then proceed past the trapping instruction.
D	Data memory break disable. The D-bit is cleared after the execution of each instruction, except for the `RFI` instruction which may set it. When set, data memory break traps are disabled. This bit allows a simple mechanism to trap on a data store and then proceed past the trapping instruction.

Data Types

The fundamental data type is a 64-bit signed integer, with the most significant bit on the left and the least significant bit on the right. All computations are performed on 64-bit signed integers. Memory accesses can be performed at the granularity of a double-word, word, halfword, or byte. Data loaded as a word, half-word, or byte is normally sign-extended to 64 bits. The load instruction has however an option to zero-extend the data loaded. Store operations write the corresponding value to memory without modification.

All memory addresses are expressed as bytes addresses. Address computation uses a 32-bit unsigned math, i.e. numbers wrap around in a 32-bit space and never cross region boundaries.

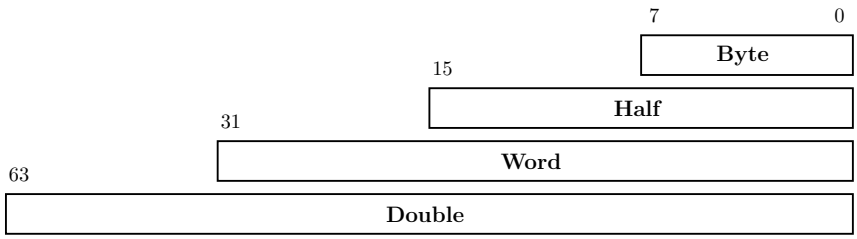


Figure 4: Data types

Byte Ordering and Alignment

Twin-64 is a big-endian machine. The following figure shows the memory access for load operations.

Figure 5: Big endian mode

The processor status register reserves a bit position, "E", for future support of mixed big and little endian memory access. Currently this bit is always set to zero.

Twin-64 enforces natural alignment for memory accesses. Byte, half-word, word, and double-word data types must be aligned to 1, 2, 4, and 8-byte boundaries respectively. If an access does not meet the required alignment, a data-alignment trap is raised.

Translation Lookaside Buffer

Virtual address translations are stored in a translation lookaside buffer (TLB). The TLB is essentially a cache of page table entries representing the virtual pages currently mapped to physical addresses. Depending on the hardware implementation, there may be a single combined TLB or separate instruction and data TLBs. Each TLB entry stores the virtual page number (VPN) along with its attributes and the corresponding physical page number (PPN). The current architecture supports a 36-bit physical address space, allowing a maximum of 64GB of memory.

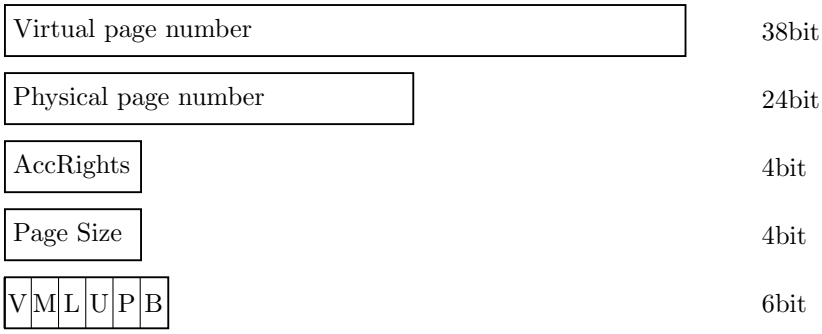


Figure 6: Tlb info

To reduce TLB processing overhead, the TLB is capable of storing virtual address ranges. There is support for a larger page sizes. The supported sizes are 4 Kbyte, 64 Kbyte, 1 MByte and 16Mbyte.

Table 3: TLB Fields

Field	**Purpose**
VPN	The virtual page number.
PPN	The physical page number.
Access Rights	Encodes the access type (read, write, and execute) and the required privilege level for the operation.
Page Size	4-bit field encoding the supported page size: `( 1U << ( pSize * 4 )) * T64_PAGE_SIZE` The pSize number 4 to 15 are reserved and result in an undefined operation. A page size is not allowed to cross region boundaries. The virtual address must be aligned with selected the page size.
V	Indicates if the entry is valid.
M	When set, the page has been modified.
L	When set, the TLB entry is locked and will not be selected during a replacement candidate search.
U	When set, the page represented by the TLB entry is not cached.
B	When set, a branch instruction executed from this page will cause a branch-taken trap. This only applies for instruction TLBs or unified TLBs.

The first virtual memory regions, region 0 to 15, are special in that they bypass the TLB entirely. A virtual address in the range 0x00_0000_0000 to 0x03_FFFF_FFFF directly maps to the physical address of the same range. The address range represents physical memory and can only be accessed in privileged mode.

Caches

Twin-64 is a cache-visible architecture that supports both unified and separate instruction and data caches. While the hardware directly manages cache line insertion and replacement, software has explicit control over flushing and purging cache line entries. Twin-64 caches are virtually indexed and physically tagged. To accelerate memory access, the cache uses the virtual address to index into the arrays in parallel with the TLB lookup. A cache hit occurs when the physical address tag in the cache matches the physical page address from the TLB.

The architecture does not mandate specific cache line sizes, cache organization, or hierarchy models. Instead, it defines instructions for cache operations, such as deletion or flushing of cache lines.

Each TLB entry indicates whether the corresponding data should be cached. For regions 0 to 15, where the TLB is bypassed, the hardware determines the caching behavior. All memory address ranges as well as the I/O address range are not cached.

Instruction Set

Twin-64 features a simple and efficient instruction set. All instructions are of fixed length, specifically a 32-bit word. The total number of instructions is relatively small, but many include execution options that make them highly versatile. Great care has been taken to ensure these options do not significantly complicate the pipeline or lengthen the data path.

Unlike CISC-style processors that support a wide variety of addressing modes, Twin-64 provides just two virtual memory addressing modes, both based on a simple baseoffset calculation. No indirection or mode requiring a memory operand to compute an address is part of the architecture.

The instruction set is organized into five groups:

- **Computational instructions:** Arithmetic, boolean, and bit-manipulation operations such as extract and deposit. Twin-64 supports only 64-bit signed arithmetic. Any numeric overflow results in a trap.
- **Immediate instructions:** Instructions for constructing immediate values up to 64 bits. Several instructions embed immediate fields within the instruction word, but a full 64-bit constant requires a sequence of two or three instructions. There are also immediate instructions that support 32-bit address arithmetic, where calculations wrap around within the 32-bit offset field and never overflow into the region identifier portion.
- **Memory reference instructions:** Load and store operations for both virtual and physical memory. These transfer data between memory and general registers in sizes of double–word, word, halfword, or byte. The design resembles a load/store architecture, but unlike strict load/store machines, several computational Twin-64 instructions allow memory operands directly. No instruction, however, both reads and writes data memory in the same operation.
- **Control flow instructions:** Both unconditional and conditional branches are supported. Unconditional branches may optionally save the return address in a general-purpose reg-

ister. Conditional branches compare two operands and transfer control if the condition is satisfied, eliminating the need for a separate compare instruction.

- **System control instructions:** Instructions for managing processor resources such as the TLB, cache, and control registers. Control registers can be accessed directly; the TLB can be updated with insert and remove operations; and the cache can be flushed or purged. Additional instructions generate traps or provide diagnostic functions for examining processor state. Most system instructions require execution in privileged mode.

The book section on the instruction set will describe each instruction group in detail, with formats, descriptions, and examples.

System Memory

Virtual Address Space

Twin-64 features a 52-bit virtual address range, organized into a 20-bit region identifier and a 32-bit offset within that region. The following figure illustrates the structure of a virtual address and how the region is selected.

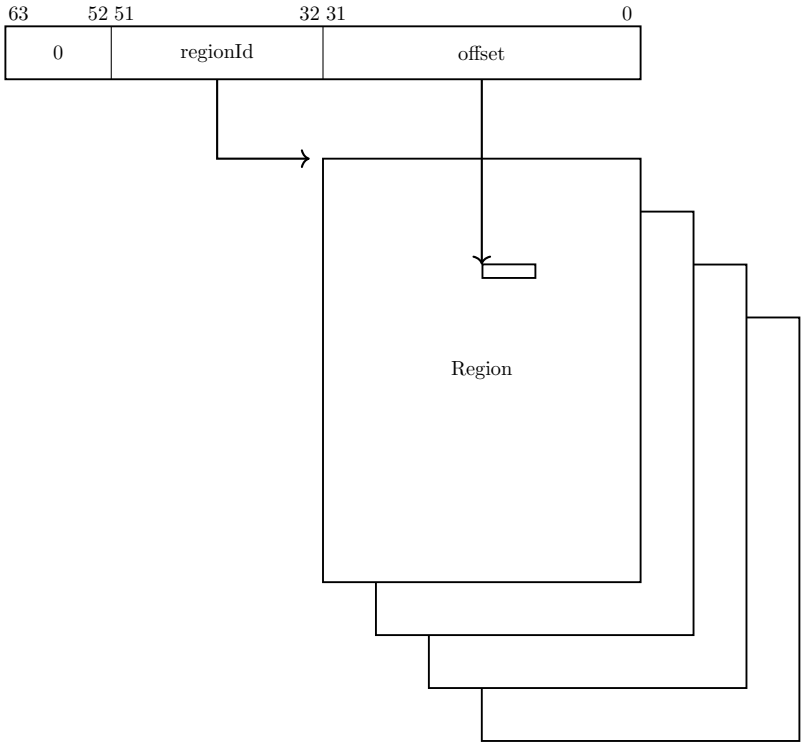


Figure 7: Virtual Memory Address Range

A virtual address is translated into a physical address. For translation purposes, the virtual address range can also be viewed as a set of virtual pages. The architected page size is 4 KBytes, with a 40-bit virtual page number and a 12-bit page offset.

Address Space Layout

Twin-64 architecture features a 36-bit physical memory address range which allows a maximum physical memory of up to 64 GBytes. The following figure depicts the overall memory layout.

Region IDs 0 to 15 represent the physical address space, with a maximum size of 64 GB. These regions are directly mapped to physical addresses, without TLB translation or access-rights

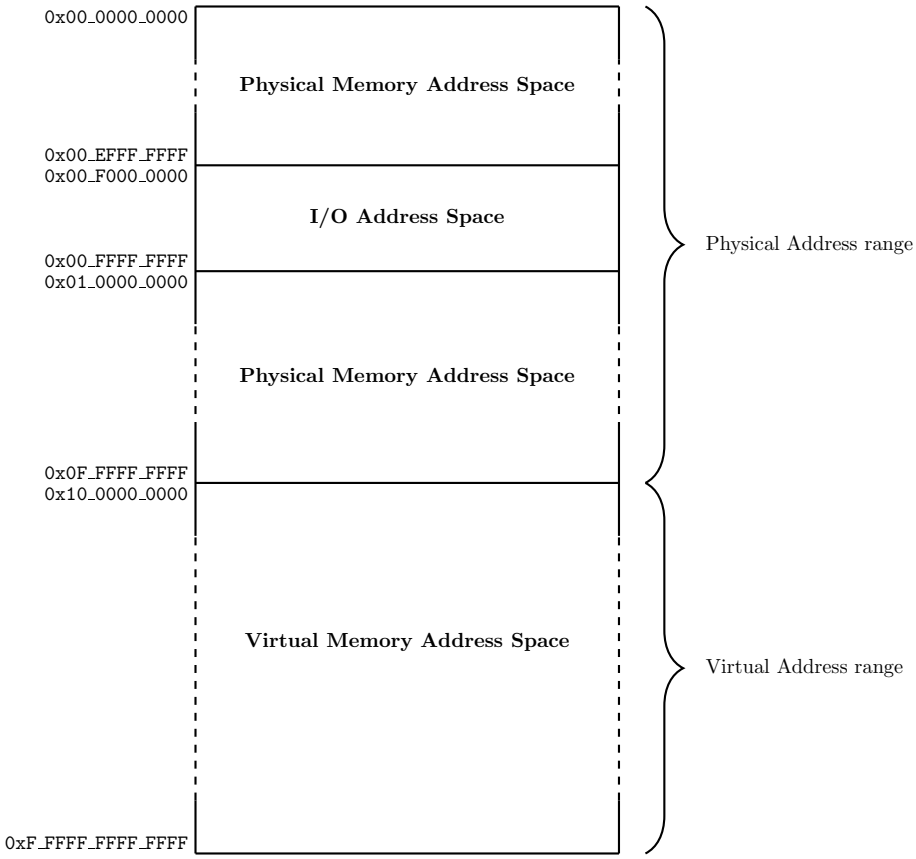


Figure 8: Memory Address Ranges

checking. Only privileged-mode code can access these regions. Control Register 0 contains a field that encodes the actual physical memory size installed in the system.

Twin-64 uses a memory-mapped I/O architecture. The I/O address space occupies the top 256 MB of the first region. I/O modules allocate their registers and storage within this range, and access to this area is uncached.

The remaining address space is dedicated to virtual memory. Virtual storage is allocated dynamically, and the TLB provides the necessary translation to physical addresses while enforcing access rights.

> **Note:** Explain the conceptual model of machine mode, supervisor mode and user mode and how this impacts address resolution. The term privileged code would need to be replaced with supervisor mode. Under evaluation. For now, there is privileged and user mode. Priv mode accesses the entire address range.

Addressing and Access Control

The CPU generates 52-bit virtual addresses. Each virtual memory access must be translated and checked for valid access rights. This chapter describes the address translation process, as well as the access-rights and protection mechanisms.

Address Translation

Each virtual memory access generated by the processor must be translated into a physical address. The translation process maps a virtual page to a physical page. Virtual pages in the first 16 regions bypass the TLB, meaning that the virtual address is identical to the physical address.

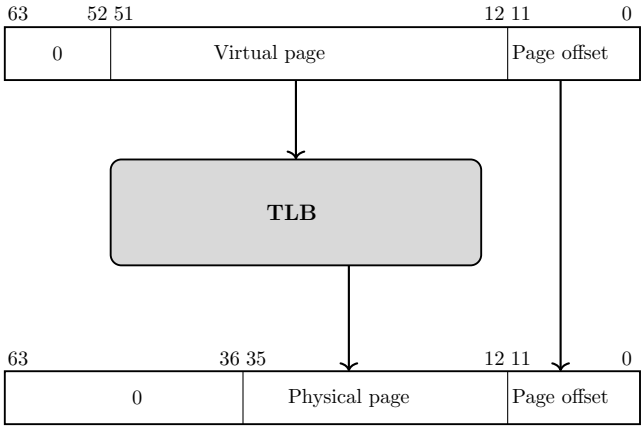


Figure 9: Address translation

Access Control

Twin-64 implements a protection model along two dimensions. The first dimension is **access control** and **privilege level**. Each page is associated with a page type: read-only, read-write, execute, or gateway. The page type is encoded in the AccType field, where 0 represents read-only access, 1 represents read-write access, 2 represents execute access, and 3 represents gateway access.

Each memory access is checked against the allowed access type defined in the page descriptor. There are two privilege levels: user mode and supervisor mode. The combination of access type and privilege level forms the access control information, which is verified for every instruction and data memory access.

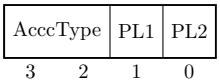


Figure 10: Access control information field

For read access, the privilege level must be at least as high as the PL1 field; for write access, it must be at least as high as PL2. Exceptions are execute and gateway pages, where write access is allowed only for privileged code. For execute access, the privilege level must be at least PL1 and no higher than PL2. If the instructions privilege level falls outside these bounds, a privilege violation trap is raised. If the page type does not match the instructions access type, an access violation trap is raised. In both cases, the instruction is aborted, and a memory protection trap occurs.

Table 4: Access type checking

Type	**read**	**write**	**execute**
`0 - read-only`	`PL <= PL1`	`not allowed`	`not allowed`
`1 - read-write`	`PL <= PL1`	`PL <= PL2`	`not allowed`
`2 - execute`	`PL <= PL1`	`PL == 0`	`PL2 <= PL <= PL1`
`3 - gateway`	`PL <= PL1`	`PL == 0`	`PL2 <= PL <= PL1`

Access Protection

The second dimension of protection is **region ID checking**. Twin-64 allows the processor to store up to eight protection ID values in the control registers `CR4` through `CR7`, which are accessible to the currently executing process. The protection ID is identical to a region ID. For each access to a region with the protection-check bit set, at least one of the protection ID registers must match the region ID; otherwise, a protection violation trap is raised.

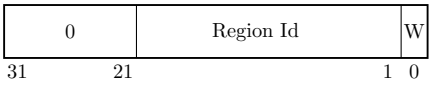


Figure 11: Protection Id

When protection ID checking is enabled, the eight protection identifiers are compared with the region ID of the target address to validate an access. The rightmost bit of each protection ID register serves as a write-disable (W) bit. When the W bit is set, that protection ID cannot be used to grant write access. If the PSW P-bit is cleared, protection ID and write-disable checks are bypassed.

Protection ID checking is typically used to enforce controlled access to shared or private memory regions. A common example is the stack data region, which is accessible at user privilege level. Since the CPU implements a global virtual memory address space, any task could potentially access the stack of another task. By enabling protection ID checking and loading the region ID into one of the protection ID registers for the stack owner process, only the corresponding user task is allowed to access its stack data region with a matching protection ID.

Note: Is the protection Id bit in the processor status word required? When we have the rule that privilege code can access anything, and user code will always be subject to protection Id checking, the bit is not needed.

Access Check Flow

The following figures summarizes the protection and access mode checking during address translation.

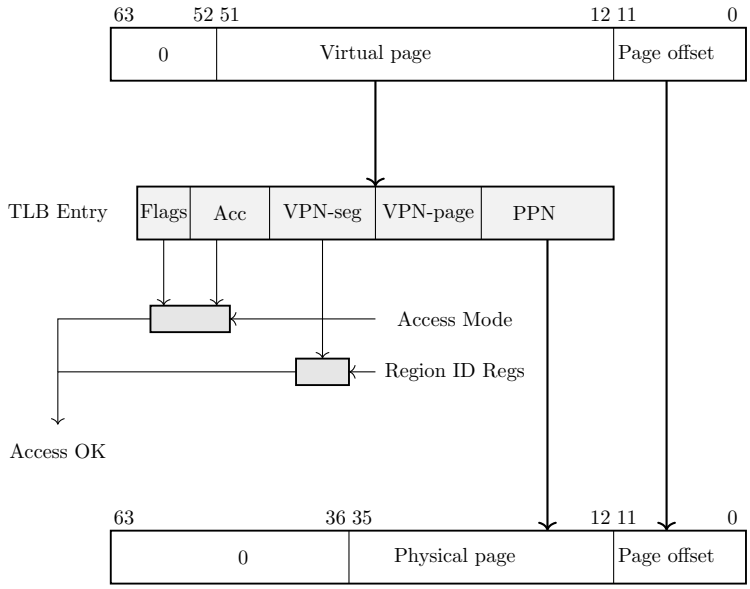


Figure 12: Address translation

For a successfully found TLB entry, the region portion of the virtual address is compared against the set of protection Id registers. If there is no match, a protection Id trap is raised. The access mode of the instruction is compared against the access rights and TLB flags of the TLB entry. If the access mode does not match an access rights trap or a privilege violation trap is raised.

Control Flow

In Twin-64, control normally proceeds to the next sequential instruction in memory unless explicitly changed by a branch or interrupted by an exception. From the programmers perspective, instructions execute in program order. Internally, however, the hardware may employ techniques such as out-of-order execution. This chapter presents the logical execution model, describing how branching and interruptions alter control flow.

Unconditional Branches

Unconditional branch instructions always redirect control flow and may optionally save the return address in a general-purpose register. The branch target address can be computed in two ways:

- **Instruction-relative branches:** The target is obtained by adding an offset to the address of the current instruction.
- **Region-relative branches:** The target is computed relative to the start of the segment (address zero of the segment).

Both methods use 32-bit unsigned arithmetic with wrap-around on overflow. For example, if the current instruction resides at `0xFFFF_FFFC` and the branch offset is `0x10`, the calculation is:

$$\texttt{0xFFFF\_FFFC} + \texttt{0x10} = \texttt{0x1\_0000\_000C}$$

Since the addition is performed modulo 2^{32}, the high-order carry is discarded. The effective branch target is therefore `0x0000_000C`.

Conditional Branches

Conditional branches combine a comparison with an instruction-relative branch. If the specified condition is true, control transfers to the branch target; otherwise, execution continues sequentially. Conditional branches are always instruction-relative.

Linkage

Certain branch instructions support procedure calls by saving the return address (the instruction immediately following the branch) into a general-purpose register. The linkage mechanism applies to both intra-segment and inter-segment branches. The saved return address consists of both the segment identifier and the instruction offset.

Cross-Region Branches

Unconditional inter-segment branches are branches to another segment. The offset is computed from the offset portion in the address specified plus the immediate value encoded in the `BE` instruction.

Privilege Level Changes

Branch instructions may alter the current privilege level. Execution is only permitted if the current privilege level satisfies the requirements of the code or data being accessed. Traditionally, privilege changes are handled through traps into the operating system kernel. While secure, this approach incurs significant overhead due to pipeline flushes and handler setup. Twin-64 provides a more efficient gateway mechanism. An unconditional branch with the `.G` gateway option may enter a gateway page, raising the privilege level. The processor then executes at the privilege level specified by the gateway page, with the PSW privilege bit updated accordingly. When returning to code in a lower-privilege page, the CPU automatically lowers the current privilege level. Each instruction fetch checks the privilege level of the code page against the PSW `P`-bit:

- If the page requires higher privilege than the PSW indicates, a privilege violation trap occurs.
- If the page requires lower privilege, execution privilege level is automatically demoted.
- If the G option on the branch instruction `B`is set, the execution privilege is set to the privilege level of the gateway page where the branch instruction resides.

Traps Associated with Branches

Branch instructions may trigger traps depending on the PSW state. For example, if the PSW `T`-bit is set and a branch is taken, a *taken-branch trap* is raised. This feature is primarily intended for debugging.

Interruptions

Twin-64 features a simple but powerful interrupt system. Interruptions in Twin-64 are divided into four classesfaults, traps, interrupts, and checksthat may occur during instruction execution.

- **Fault** - The current instruction cannot be carried out due to a system problem, such as the absence of a page from main memory. After the system problem has been corrected, the faulting instruction should execute as intended. Faults are synchronous with respect to the instruction stream.
- **Trap** - There are two kind of traps. The types are traps raised because the function requested by the current instruction cannot or should not be carried out. A good example is the arithmetic signed overflow trap. Such an instruction is normally not re-executed. The second type of traps are raised so that the system can intervene before the or after the instruction is executed. An example for this type of trap is the debugging breakpoint trap. Traps are synchronous with respect to the instruction stream.
- **Interrupt** - An external entity such as an I/O module requires attention. Interrupts are asynchronous with respect to the instruction stream.
- **Check** - The processor has detected an internal fault or malfunction. Checks can be either synchronous or asynchronous with respect to the instruction stream.

All four classes of interruptions are handled in the same way.

Interrupt Handling

Handling of an interrupt is implemented as a fast context switch. When an interruption occurs, the hardware saves the PSW at the time of the interrupt to the IPSW control register. Depending on the interrupt type, this can be the current instruction of the next instruction. All status bits, which are part of the PSW, are saved too. The hardware will then set the PSW status bits as follows. The M bit is set if the interruption is a high priority machine check. All other bits are set to zero. Depending on the trap type, additional data is saved to the interrupt argument control registers IARG0 and IARG1.

The control register `IVA` holds the virtual address of the interrupt vector table. Execution of the interrupt handler begins at the address computed using the trap number:

$$\texttt{IVA} + (32 * \texttt{trap number})$$

Each entry consists of 8 instructions that can be executed. It is the responsibility of interruption handlers to unmask external interrupts as soon as possible, so as to minimize interrupt latency. The interruptions are categorized into four groups.

Example. Suppose the interrupt vector table base address is:

$$\texttt{IVA} = \texttt{0x0000\_1000}$$

Now, if a **trap number 5** (Illegal Instruction) occurs, the handler entry point is calculated as:

$$\texttt{0x0000\_1000} + (32 * 5) = \texttt{0x0000\_10A0}$$

Execution of the interrupt handler starts at `0x0000_10A0`.

Interrupt Vector Table

Table 5: Trap Table

Trap	Name	Instruction Adr	Arg0	Arg1
1	Machine Check	current instruction	check type	-
2	Power Failure	current instruction	-	-
3	Recovery Counter	current instruction	-	-
4	External Interrupt	next instruction	-	-
5	Illegal instruction	current instruction	instruction	-
6	Privileged operation	current instruction	instruction	-
7	Privileged register	current instruction	instruction	-
8	Integer Overflow	current instruction	instruction	-
9	Instruction TLB miss	current instruction	-	-
10	Non-access Instruction TLB miss	current instruction	-	-
11	Instruction memory protection	current instruction	-	-
12	Data TLB miss	current instruction	instruction	target address
13	Non-access Data TLB miss	current instruction	instruction	target address
14	Data memory access rights	current instruction	instruction	target address
15	Data memory protection	current instruction	instruction	target address
16	Page reference	current instruction	instruction	target address
17	Alignment	current instruction	instruction	target address
18	Break Instruction	current instruction	instruction	-
19	Taken branch	next instruction	instruction	-

Part III

Instruction Set

Instruction Set

This part of the book presents the Twin-64 instruction set. Following the RISC design philosophy, it features a fixed-length format with a small number of regular instruction types. The total number of instructions is relatively small; however, many instructions encode execution options that make them highly versatile. Great care is taken to ensure that these options do not significantly complicate the pipeline or increase the overall data path length. Instead of supporting a wide variety of addressing modes, as commonly found in CISC-style CPUs, Twin-64 provides two virtual memory addressing modes based on a simple base-offset calculation. No indirection or addressing mode that requires reading a memory operand to compute an address is part of the architecture.

The instruction set is divided into five groups.

- **Memory reference instructions:** Load and store operations for virtual and physical memory.
- **Immediate instructions:** Building immediate values up to 64 bits.
- **Control flow instructions:** Conditional and unconditional branches.
- **Computational instructions:** Arithmetic, boolean, and bit operations such as extract and deposit.
- **System control instructions:** Managing hardware resources such as caches and TLBs, register movement, traps, and interrupt handling.

Arithmetic Instructions

The arithmetic instructions operate on two signed 64-bit operands and store the result in the target register. There are two instruction layouts. The immediate operand instruction format adds the sign-extended 15-bit value to `RegB`. The register instruction format uses two registers as input operands, performs the operation, and stores the result in a third register.

The `ADD` and `SUB` instructions perform signed 64-bit integer arithmetic, with numeric overflow triggering an overflow trap. There are no operations on unsigned quantities. The `SHLxA` and `SHRxA` instructions perform an arithmetic left or right shift on the input operand and add another operand to the shifted result, storing the final value in the target register. The `CMP` instruction compares two registers for a condition and stores the result in the target register.

Bit Operations Instructions

The bit operation instructions fall into two groups. The `AND`, `OR`, and `XOR` instructions perform the logical operations indicated by their names. Like the arithmetic instructions, they have both an immediate instruction format and a register instruction format.

The second group implements bit manipulation operations. The `EXTR` instruction extracts a bit field from a register and places it in another register; optionally, the extracted field can be sign-extended. The `DEP` instruction deposits a bit field into a target register. The `DSR` instruction concatenates two general-purpose registers, shifts the resulting 128-bit quantity to the right, and stores the lower 64 bits in the target register.

Immediate Instructions

The `LDI` and `ADDIL` instructions are used for forming large constants. With a 32-bit instruction word, it is not possible to encode even a full 32-bit constant in a single instruction. The `LDI` instruction includes a qualifier to load a constant value at a defined position in the target register. Using a short sequence of instructions, any 64-bit value can be loaded.

The `ADDIL` instruction adds the upper portion of a 32-bit immediate value to the lower 32-bit half of a general register. This instruction is primarily used for address arithmetic, enabling access to any location within a segment. The `LDO` instruction loads an offset into a general register. The address is formed by adding the signed immediate value to a base register. Address arithmetic is performed as unsigned 32-bit arithmetic, wrapping around within the 32-bit range.

Memory Reference Instructions

Twin-64 is a register-memory architecture. It supports the common load/store instructions as well as instructions that combine an operation with a memory reference. Memory reference instructions feature two addressing modes. The first is the base register plus offset mode, and the second is the base register plus register index mode.

The indexed instruction format loads the content of the memory address formed by adding the scaled 13-bit immediate field to the base register `RegB`, and adds the fetched data value to `RegA`. The `dw` field specifies the data width: a `dw` value of 0 loads an 8-bit value, 1 loads a 16-bit value, 2 loads a 32-bit value, and 3 loads a 64-bit value. In all cases, the fetched value is sign-extended to 64 bits. Additionally, the `dw` field scales the offset by left-shifting the immediate field according to the data width. For example, a `dw` value of 3 loads a double word from an address formed by shifting the 13-bit offset by three bits before adding it to the base register.

The register-indexed instruction format loads the content of the memory address formed by adding `RegX` to the base register `RegB`. The `dw` field specifies the data width, as in the indexed instruction format. The offset in `RegX` is interpreted as a byte offset, independent of the `dw` value. In both instruction formats, the fetched data is sign-extended to 64 bits.

The `LD` and `ST` instructions are the standard load and store instructions. In addition to the two addressing modes described above, they also support base register modification.

The `LDR` and `STC` instructions provide the base support for a pipeline-friendly method for semaphore operations. They only support the indexed instruction mode with double-word access, and the base register modification option is not available.

The `ADD`, `SUB`, `AND`, `OR`, `XOR`, and `CMP` instructions perform the respective operations as described above, with one operand fetched from memory using the indexed addressing modes.

Control Flow Instructions

Control flow is implemented through a set of branch instructions, which can be classified into unconditional and conditional branches.

Unconditional branches fetch the next instruction address from the computed branch target. The `B` and `BR` instructions perform an instruction address relative branch. The `BV` instruction performs a vectored branch. The content of a general register is added to a base register to form the branch target. The `BE` instruction is used to perform a cross-region branch. All instructions can optionally store the return address in a general register.

The `BB`, `CBR`, `MBR`, and `ABR` instructions implement conditional branching. If the specified condition is met, a branch to the target address is performed. The target address is always a local address, with the offset being instruction-address relative. Conditional branches adopt a static prediction model where forward branches are assumed not taken, while backward branches are assumed taken.

All branch address arithmetic is performed as 32-bit unsigned arithmetic with wraparound. Adding branch offsets will not overflow into the segment Id portion of the address.

System Instructions

The final instruction group is the **special/system** instruction group. The system control instructions manage CPU resources such as the TLB, cache, and other hardware components. Most instructions in this group require the processor to run in privileged mode.

The `MFCR` and `MTCR` instructions are used to transfer data to and from a control register. Access to some control registers requires privileged mode. The `MFIA` instruction accesses the current instruction address, which is important for position-independent code. The `RSM` and `SSM` instructions allow setting or clearing an individual accessible bit in the status portion of the processor state.

The `LDPA`, `PRBR`, and `PRBW` instructions are used to obtain information about the virtual memory system. The `LDPA` instruction returns the physical address of the virtual page, if present in memory. The `PRBx` instructions test whether the desired access mode to an address is allowed.

The translation lookaside buffers and caches are managed by the `IxTLB`, `PxTLB`, `FxCA`, and `PxCA` instructions. The `IxTLB` and `PxTLB` instructions insert or remove translations from the TLB. The `FxCA` and `PxCA` instructions manage the instruction and data caches, providing options to flush or purge a cache line. There is no instruction to insert data into a cache, as cache insertion is fully controlled by hardware.

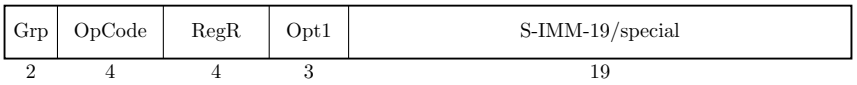
The `RFI` instruction restores the processor state from the control registers. Optionally, general registers stored in reserved control registers will also be restored.

The `DIAG` instruction issues hardware-specific implementation commands. An example is the memory barrier command, which ensures that all memory access operations complete before subsequent instructions execute.

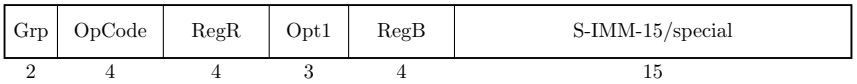
The `TRAP` instruction raises a software exception and is typically used to implement a debugging breakpoint.

Instruction Formats

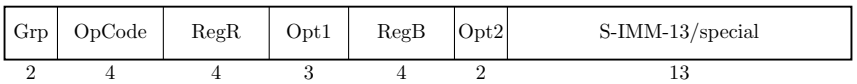
Twin-64 employs a small number of main instruction formats, which are classified according to the number of register operands and the length of the immediate value field. The **signed immediate S19 / special (S19)** format provides one register operand and a 19-bit signed immediate field. This format is typically used for branch instructions.



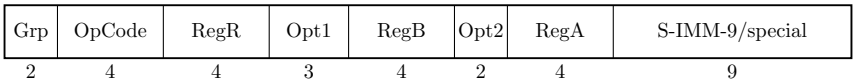
The **signed immediate S15 / special (S15)** instruction format provides two register operands and a 15-bit signed immediate field. This format is used both by conditional branch instructions and by instructions that operate on a register and an immediate value.



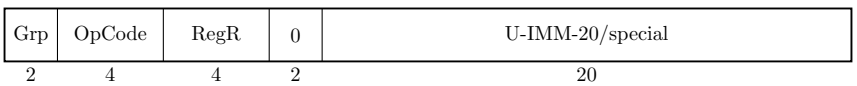
The **signed immediate S13 / special (S13)** instruction format provides two registers, an additional option field, and a 13-bit signed immediate field. Some instructions use the S-IMM-13 field for special encodings instead of a signed value. This format is used by memory reference instructions, where the address is formed by a base register plus a 13-bit signed offset.



The **register / signed immediate S9 / special (S9)** instruction format provides three registers, an additional option field, and a 9-bit signed immediate field. Some instructions use the S-IMM-9 field for special encodings instead of a signed value. This instruction format is used for all computational instructions that require three registers.



The **unsigned immediate U20 (U20)** format is used by the immediate instruction group to provide a 20-bit unsigned value. This value is then placed at specific positions in the target register `Reg R`.



Instruction Set Descriptions

This chapter provides a detailed description of the Twin-64 instruction set. Each instruction entry includes:

- The instruction word layout.
- A concise description of the instruction.
- A high-level pseudo code representation of its operation.

Instruction operations are expressed using C-style pseudo code. The routines and conventions used are listed in the appendix. Registers are referenced by class and index:

- `GR[5]` general register 5
- `CR[1]` control register 1
- Some registers have dedicated names, e.g., the shift amount control register is `SAR`.

Subfields of an instruction are accessed using a dot and the field name in square brackets. For example:

- The interrupt enable bit in the PSW register: `PSW.[I]`.

Instruction options are specified in assembler syntax by a dot followed by one or more characters, each representing a defined option. For example:

- `AND.N ...` the `AND` instruction with the negate result option.
- The order of option characters does not matter; all defined options are listed in the relevant character set.

Fields marked as reserved are intended for future enhancements and should always be set to zero. Any other value in a reserved field results in undefined behavior.

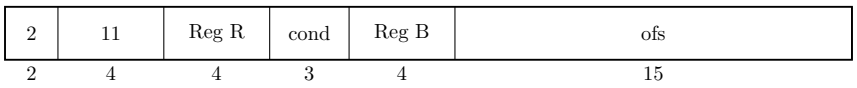
ABR - Add and branch

Syntax

```
ABR.<cond> RegR, RegB, target
```

Format

The ABR instruction uses the following instruction format:



Description

The `ABR` instruction adds `RegB` to `RegR` and stores the result in `RegR`. If the condition specified in the `cond` field is satisfied, execution continues at the current instruction plus the offset; otherwise, execution proceeds with the next instruction.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == 0`
1	LT	`RegR < 0`
2	GT	`RegR > 0`
3	EV	`RegR.[0] == 0`
4	NE	`RegR != 0`
5	GE	`RegR >= 0`
6	LE	`RegR <= 0`
7	OD	`RegR.[0] != 0`

Operation

```
RegR <- RegR + RegB;
if ( cond( RegR ))
    PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], ofs << 2 );
else
    PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], 4 );
```

Exceptions

Integer Overflow trap.

Notes

See also the instruction notes on comparison and branch condition encoding in the appendix.

ADD Integer Addition

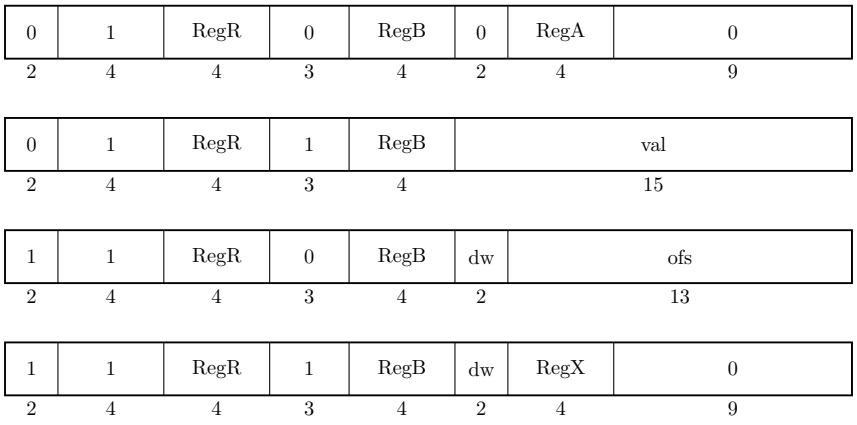
Syntax

```
ADD RegR, RegB, RegA
ADD RegR, RegB, val

ADD[.D/W/H/B] RegR, ofs ( RegB )
ADD[.D/W/H/B] RegR, RegX ( RegB )
```

Formats

The ADD instruction uses the following instruction formats:



Description

The `ADD` instruction adds two signed 64-bit operands and stores the result in the target register `RegR`. This instruction supports the immediate, register, and both memory-reference formats. An arithmetic overflow triggers a numeric overflow trap, and indexed instruction formats may cause a memory-reference related trap.

Operation

```
RegR <- RegB + RegA;                              (S9 )
RegR <- RegB + immOperand( instr );               (S15)
RegR <- RegR + memOperandImmIndexed( instr );     (S13)
RegR <- RegR + memOpwerandRegIndexed( instr );    (S9 )
```

Exceptions

Integer Overflow Trap
Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

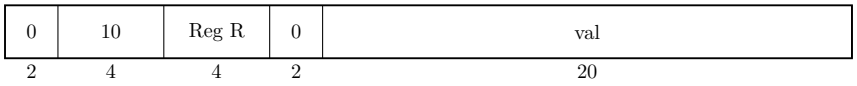
ADDIL - Add immediate left

Syntax

```
ADDIL RegR, val
```

Format

The ADDIL instruction uses the uses the following instruction formats.



Description

The `ADDIL` instruction adds an immediate value to a general register using 32-bit unsigned arithmetic. The 20-bit immediate value is shifted left by 12 bits (padded with zeros on the right) before being added to `RegR`. The resulting value is stored in the general register `GR1`. Any overflow that occurs during the addition is ignored.

Operation

```
RegR <- addOfs32( RegR, ( val << 12 ));
```

Exceptions

None.

Notes

None.

AND - Boolean Operation

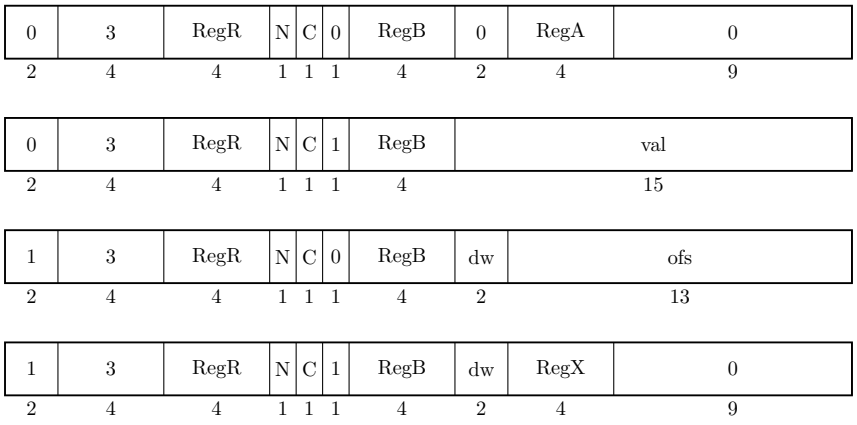
Syntax

```
AND[.C/N] RegR, RegB, RegA
AND[.C/N] RegR, RegB, val

AND[.C/N/D/W/H/B] RegR, ofs( RegB )
AND[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The AND instruction uses uses the following instruction formats.



Description

The `AND` instruction performs a bitwise AND operation on two 64-bit operands and stores the result in the target register `RegR`. The second operand, `RegB` or the loaded content, can be negated by setting the "C" bit, and the result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The Boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-reference related trap.

Operation

```
tmp <- RegA;                              (S9)
tmp <- immOperand( instr );               (S15)
tmp <- memOperandImmIndexed( instr );     (S13)
tmp <- memOpwerandRegIndexed( instr );    (S9 )

if ( instr.[C] ) tmp <- not( tmp );
RegR <- RegB & tmp;
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

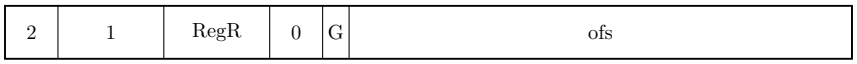
B Branch Unconditional

Syntax

```
B[.G] target [, RegR ]
```

Format

The `B` instruction uses the following instruction format:



Description

The `B` instruction performs an unconditional instruction address relative branch. The target address is computed by sign-extending the immediate offset, shifting it left by two bits, and adding the result to the current instruction address. The return link is stored in `RegR`.

The `.G` flag enables the *gateway branch* option. In this case, the processors privilege level may be raised to the privilege level stored in the TLB entry of the page from which the gateway instruction is fetched. The change is only performed if it results in a higher privilege level; otherwise, the current privilege level remains unchanged. Execution then continues at the computed target address with the (possibly new) privilege level.

Operation

```
if ( instr.[RegR] != 0 ) {

        RegR.[51:32]<- PSW.[IA-REG];
        RegR.[31:0]  <- addOfs32( PSW.[IA-OFS], 4 );
}

if ( instr.[G] ) {

    if ( TLB[PSW.[IA-OFS]].[X] > PSW.[X] )
        PSW.[X] <- TLB[PSW.[IA]].[X];
}

PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], ofs << 2 );
```

Exceptions

Instruction Privilege Violation Trap
Instruction TLB Miss Trap
Taken Branch Trap

Notes

None.

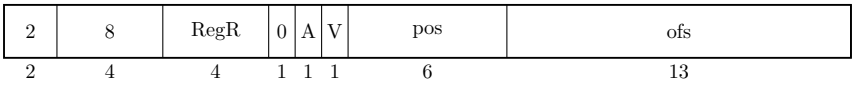
BB Branch on Bit

Syntax

```
BB[.T/F] ofs, pos
BB[.T/F] ofs, SAR
```

Format

The BB instruction uses the following instruction format:



Description

The BB instruction tests a bit in register RegR. The bit position can be specified directly by the pos field, or indirectly through the Shift Amount Register (SAR) if the A option is set.

If the V bit is set, the test succeeds when the selected bit is 1. If the V bit is clear, the test succeeds when the selected bit is 0.

When the condition is satisfied, execution branches to the target address, computed by sign-extending the offset, shifting it left by two bits, and adding it to the current instruction address. If the condition is not satisfied, execution continues with the next sequential instruction.

Operation

```
if ( instr.[A] )  pos <- SAR;
else              pos <- instr.[pos];

bVal <- testBit( RegR, pos );

cond <- ( instr.[V] ? ( bVal == 1 ) : ( bVal == 0 ));

if ( cond ) PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], ofs << 2 );
else        PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], 4 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

Notes

None.

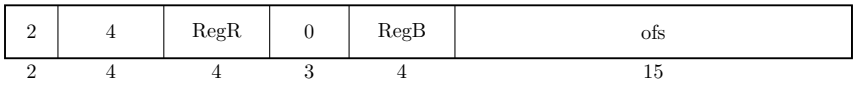
BE - Branch External

Syntax

```
BE [ ofs ] ( RegB ) [, RegR ]
```

Format

The `BE` instruction uses the following instruction format:



Description

The `BE` instruction performs an unconditional branch to another region. `RegB` contains a target region and offset base to which the sign extended offset shifted by two is added. The return link is stored in `RegR`.

Because `BE` is a region base relative branch, branching to a page with a different privilege level is possible. Branching from a lower privilege level to a higher one triggers an instruction protection trap. Branching from a higher privilege to a lower privilege level updates the privilege level in the processor status register accordingly. If no change is required, the privilege level remains unchanged.

Operation

```
if ( instr.[RegR] != 0 ) {

    RegR.[51:32]<- PSW.[IA-REG];
    RegR.[31:0]  <- addOfs32( PSW.[IA-OFS], 4 );
}

PSW.[IA-REG] <- RegB.[51:32];
PSW.[IA-OFS] <- addOfs32( RegB, RegX << 2 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

Notes

None.

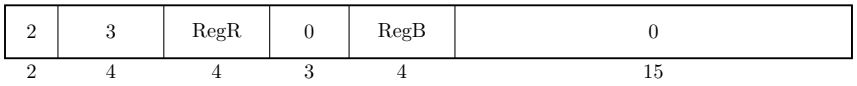
BR Branch Register

Syntax

```
BR RegB [, RegR ]
```

Format

The `BR` instruction uses the following instruction format:



Description

The `BR` instruction performs an unconditional branch to a target address computed relative to the current instruction address segment (`IA`). The target address is formed by taking the contents of `RegB`, shifting it left by two bits, and adding it to the `IA` offset portion. The return link is stored in `RegR`.

Operation

```
if ( instr.[RegR] != 0 ) {

    RegR.[51:32]<- PSW.[IA-REG];
    RegR.[31:0]  <- addOfs32( PSW.[IA-OFS], 4 );
}

PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], RegB.[31:0] << 2 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

Notes

None.

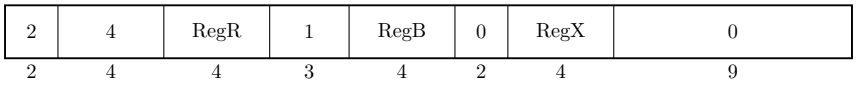
BV - Branch Vectored

Syntax

```
BV [ RegB ] ( RegR ) [ , RegR ]
```

Format

The `BV` instruction uses the following instruction format:



Description

The `BV` instruction performs an unconditional region relative branch. The content of `RegX` is shifted by two and added to `RegB`, forming the target offset in the current region. The return link is stored in `RegR`.

Because `BV` is a region base relative branch, branching to a page with a different privilege level is possible. Branching from a lower privilege level to a higher one triggers an instruction protection trap. Branching from a higher privilege to a lower privilege level updates the privilege level in the processor status register accordingly. If no change is required, the privilege level remains unchanged.

Operation

```
if ( instr.[RegR] != 0 ) {

    RegR.[51:32] <- PSW.[IA-REG];
    RegR.[31:0]  <- addOfs32( PSW.[IA-OFS], 4 );
}

PSW.[IA-OFS] <- addOfs32( RegB, RegX << 2 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

Notes

None.

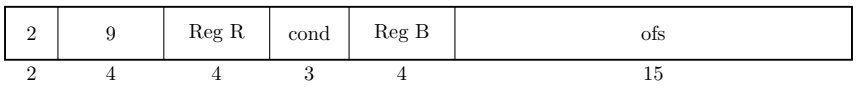
CBR - Compare and Branch

Syntax

```
CBR.<cond> RegR, RegB, target
```

Format

The CBR instruction uses the following instruction format:



Description

The `CBR` instruction compares the contents of `RegB` with `RegR` using the condition specified in the `cond` field. If the condition is satisfied, control is transferred to the instruction address computed by adding the sign-extended and word-aligned offset to the current instruction address. Otherwise, execution continues with the next sequential instruction. The comparison operation uses the same signed 64-bit comparator as the `CMP` instruction, and neither source register is modified.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == RegB`
1	LT	`RegR < RegB`
2	GT	`RegR > RegB`
3	reserved	
4	NE	`RegR != RegB`
5	GE	`RegR >= RegB`
6	LE	`RegR <= RegB`
7	reserved	

Operation

```
if ( compare( cond, RegR, RegB ))
   PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], ( ofs << 2 ));
else
   PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], 4 );
```

Exceptions

Taken Branch Tap.
Instruction TLB Miss.

Notes

See also the instruction notes on comparison and branch condition encoding in the appendix.

CMP - Compare

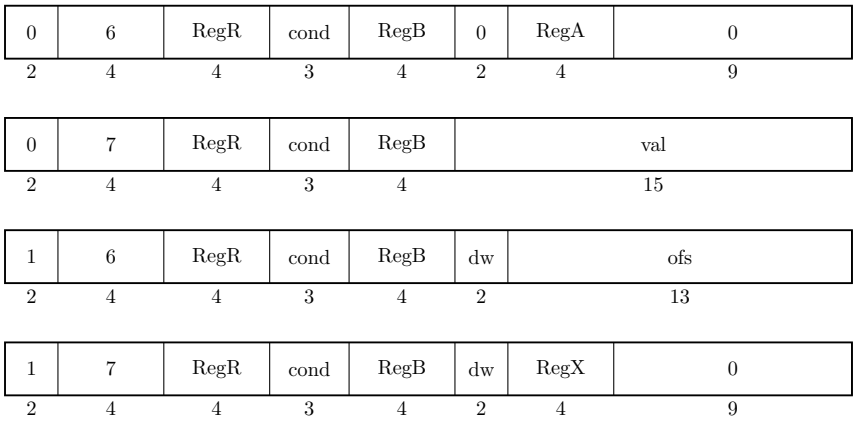
Syntax

```
CMP RegR, RegB, RegA
CMP RegR, RegB, val

CMP[.D/W/H/B] RegR, ofs ( RegB )
CMP[.D/W/H/B] RegR, RegX ( RegB )
```

Format

The CMP instruction uses the following instruction formats.



Description

The `CMP` instruction compares two signed 64-bit operands. In register mode, it compares `RegB` and `RegA`; in immediate mode, it compares `RegB` and the immediate value; and in memory-reference formats, it compares `RegR` with the memory operand. The operands are called `op1` and `op2`. The result of the comparison is stored in RegR. The instruction does not cause an arithmetic overflow but indexed instruction formats may cause a memory-referencerelated trap.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == RegB`
1	LT	`RegR < RegB`
2	NE	`RegR != RegB`
3	GE	`RegR >= RegB`
4	LE	`RegR <= RegB`
5	undefined	
6	GT	`RegR > RegB`
7	undefined	

Operation

```
RegR <- compare( cond, RegB, RegA );                  (S9 )
RegR <- compare( cond, RegB, immOperand());           (S15)
RegR <- compare( cond, RegR, memOperandImmIndexed()); (S13)
RegR <- compare( cond, RegR, memOperandRegIndexed()); (S9 )
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

See also the instruction notes on comparison and branch condition encoding in the appendix.

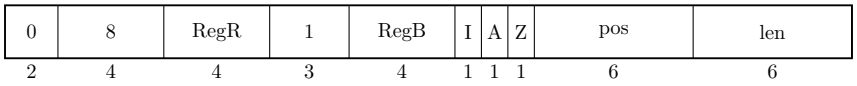
DEP - Deposit Bit Field

Syntax

```
DEP[.Z] RegR, val, pos, len
DEP[.Z] RegR, RegB, SAR, len
```

Format

The `DEP` instruction uses the special-13 instruction format.



Description

The `DEP` instruction deposits a bit field from general register `RegB` into `RegR` at the specified position.

The rightmost bit of the target field is specified by the `pos` instruction field, or by the Shift Amount Register (`SAR`) if the `A` bit is set. The length of the field is specified by the `len` instruction field.

If the `Z` bit is set, the target register `RegR` is cleared before depositing the field. If the `I` bit is set, the value to deposit is taken from the RegB field rather than a register in `RegB`.

Operation

```
if ( instr.[Z] ) RegR <- 0;

if ( instr.[A] ) pos <- SAR;
else             pos <- instr.[pos];

if ( instr.[I] )
    RegR <- deposit( RegR, instr.[RegB], pos, instr.[len] );
else
    RegR <- deposit( RegR, RegB, pos, instr.[len] );
```

Exceptions

None

Notes

None

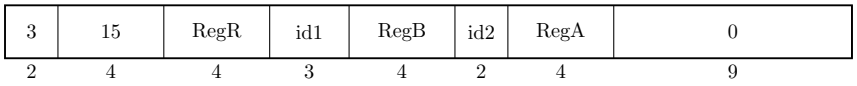
DIAG - Diagnose Instruction

Syntax

```
DIAG id, RegB, RegA
```

Format

The `DIAG` instruction uses the following instruction format:



Description

The `DIAG` instruction provides a mechanism for diagnostic and system level operations.

The concatenated `id1` and `id2` fields specify the diagnostic function. The `RegB` and `RegA` fields may provide operands or data required by the diagnostic function. Any result is returned in `RegR`.

Operation

```
RegR <- diagOperation( concat( id1, id2 ), RegB, RegA );
```

Exceptions

Exceptions depend on diagnostic function and operands.

Notes

See the appendix for the available diagnostic operation codes.

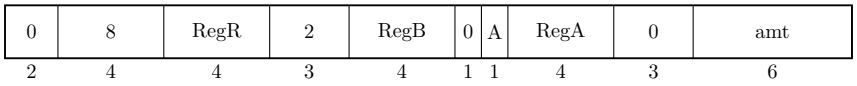
DSR Double Shift Right

Syntax

```
DSR RegR, RegB, RegA, amt
DSR RegR, RegB, RegA, SAR
```

Format

The `DSR` instruction uses the following instruction format:



Description

The `DSR` instruction concatenates the contents of registers `RegB` (left) and `RegA` (right) and performs a right shift operation. The lower 64 bits of the shifted result are stored in `RegR`.

The shift amount is specified by the `amt` instruction field, or by the Shift Amount Register (`SAR`) if the `A` bit is set.

Operation

```
if ( instr.[A] ) shAmt <- SAR;
else             shAmt <- instr.[amt];

RegR <- doubleShiftR( RegB, RegA, shAmt );
```

Exceptions

None

Notes

None.

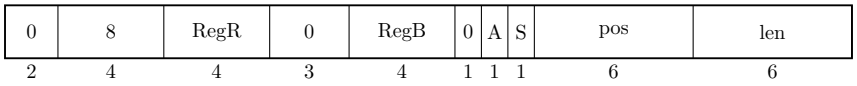
EXTR - Extract Bit Field

Syntax

```
EXTR[.S] RegR, RegB, pos, len
EXTR[.S] RegR, RegB, SAR, len
```

Format

The `EXTR` instruction uses the following instruction format:



Description

The `EXTR` instruction extracts a bit field from general register `RegB`.

The rightmost bit of the field is specified by the `pos` instruction field. If the `A` bit is set, the position value is taken from the Shift Amount Register (`SAR`) instead of the instruction field.

The length of the field is specified by the `len` instruction field. The extracted bit field is stored right-justified in `RegR`. If the `S` bit is set, the extracted value is sign-extended.

Operation

```
if ( instr.[A] ) tmpPos <- SAR;
else             tmpPos <- instr.[pos];

RegR <- extractField( RegB, tmpPos, instr.[len] );

if ( instr.[S] ) RegR <- signExtend( RegR, instr.[len] );
```

Exceptions

None

Notes

None

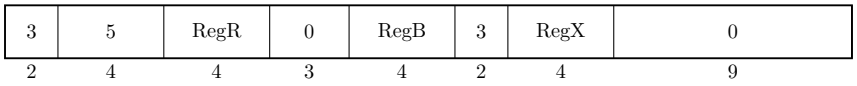
FxCA - flush from data cache

Syntax

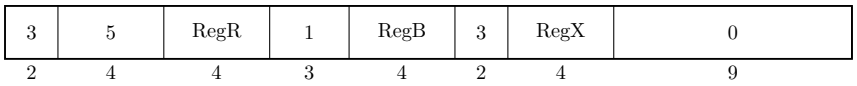
```
FICA RegR, [ RegX ] ( RegB )
FDCA RegR, [ RegX ] ( RegB )
```

Format

The `FICA` instruction uses the following instruction format.



The `FDCA` instruction uses the following instruction format.



Description

The `FxCA` instruction flushes a cache line, selected by the effective address in `RegB`. The instruction supports register indexed format. This is a privileged instruction.

Operation

```
RegR <- flushFromCache( concat( RegB.[51:32], addOfs32( RegB,
    RegX )));
```

Exceptions

Privilege violation trap.

Notes

None.

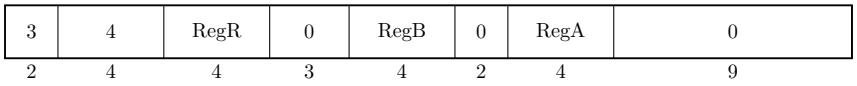
IxTLB Insert TLB Entry

Syntax

```
IITLB RegR, RegB, RegX
IDTLB RegR, RegB, RegX
```

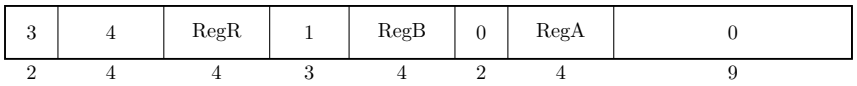
Format

The `IITLB` instruction uses the following instruction format.



3	4	RegR	0	RegB	0	RegA	0
2	4	4	3	4	2	4	9

The `IDTLB` instruction uses the following instruction format.



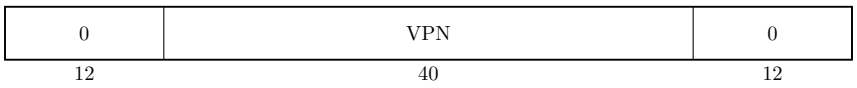
3	4	RegR	1	RegB	0	RegA	0
2	4	4	3	4	2	4	9

Description

The `IxTLB` instruction inserts a new entry into the translation lookaside buffer (xTLB). The virtual address is stored in `RegB` and additional control information is provided by `RegA`. This is a privileged instruction.

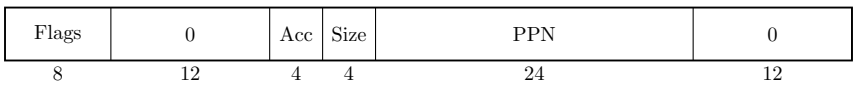
TLB Arguments

`RegB` and `RegA` contain the arguments passed to the TLB subsystem. `RegB` contains the virtual address. The address is always o a page boundary and in the case of address ranges marks the starting page.



0	VPN	0
12	40	12

`RegA` contains the flags, access rights, page size and the physical address of the address range. The appendix contains the definitions of the TLB info word.



Flags	0	Acc	Size	PPN	0
8	12	4	4	24	12

Operation

```
RegR <- insertIntoTlb( RegB, RegA );
```

Exceptions

Privilege Violation Trap.

Notes

None.

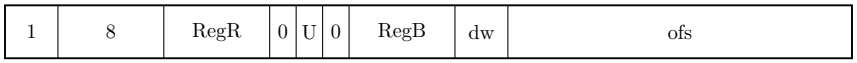
LD Load Register

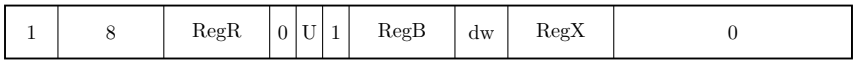
Syntax

```
LD[.D/W/H/B/U] RegR, SignedImmed13( RegB )
LD[.D/W/H/B/U] RegR, RegX( RegB )
```

Format

The `LD` instruction uses the following instruction formats:





Description

The `LD` instruction loads a data value from memory into a general-purpose register. Two formats are available: the immediate-offset format and the register-indexed format. In the immediate-offset format, the signed immediate offset is shifted right by the `dw` field of the instruction and added to `RegB` to form the effective address. In the register-indexed format, `RegX` is added to `RegB` to form the effective address.

If the data loaded is a byte, half-word or a word, the data is sign-extended. If the `U` option is set, the data is zero-extended instead.

Operation

```
RegR <- memOperandImmIndexed( instr ); (S13)
RegR <- memOperandRegIndexed( instr ); (S9)
```

Exceptions

- Data Alignment Trap
- Memory Access Violation Trap
- Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

None.

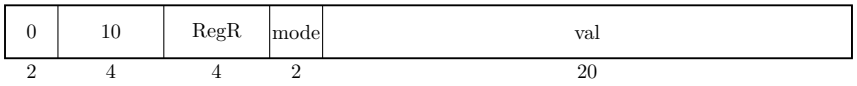
LDIL Load Immediate Left

Syntax

`LDIL[.L/.M/.U] RegR, val`

Format

The `LDIL` instruction uses the following instruction format:



Description

The `LDIL` instruction deposits an immediate value into general register `RegR`. The optional suffix determines how the immediate value is positioned within the destination register. If no suffix is specified, the default is `.L` mode (mode value = 1).

- `.L` (mode = 1) Deposits 20 bits of `val` into bits [31..12] of `RegR`. RegR is cleared before the operation.
- `.M` (mode = 2) Deposits 20 bits of `val` into bits [51..32] of `RegR`. RegR is not cleared before the operation.
- `.U` (mode = 3) Deposits 12 bits of `val` into bits [63..52] of `RegR`. RegR is not cleared before the operation.

A mode value of zero is reserved and must not be used.

Operation

```
RegR <- ((val & 0xFFFFF) << 12 );                 // .L mode (1)
RegR <- deposit( RegR, (val & 0xFFFFF), 32, 20 ); // .M mode (2)
RegR <- deposit( RegR, (val & 0xFFF), 52, 12 );   // .U mode (3)
```

Exceptions

None.

Notes

The `LDIL` instruction is used to construct large values in a register. The `.L` variant clears the destination register before depositing the immediate value. It is the default option for the `LDIL` instruction. This form is intended for building larger offsets for indexed addressing modes, where an addressing instruction adds its own 12-bit offset in the low bits. Together, these operations allow the construction of any 32-bit offset.

In contrast, the `.M` and `.U` variants deposit their immediate values without clearing the register, preserving the remaining bits. Using a combination of these modes, a full 64-bit value can be assembled in two to four instructions.

For example constructing a 52-bit value, the instruction sequence

```
LDIL   R1, L%0xA_BCDE_1234_5678  ; load    0x1234_5000 into R1
ADDIL  R1, R%0xA_BCDE_1234_5678  ; add     0x678       into R1
LDIL.M R1, M%0xA_BCDE_1234_5678  ; deposit 0xA_BCDE    into R1
```

loads the constant `0xA_BCDE_1234_5678` into R1.

LDO Load Offset

Syntax

`LDO [.B/H/W/D] RegR, ofs(RegB)`

Format

The `LDO` instruction uses the following instruction formats:

Description

The `LDO` instruction loads an offset into a general register. Two formats are available: the immediate-offset format and the register-indexed format. In the immediate-offset format, the signed immediate offset is shifted right by the `dw` field of the instruction and added to `RegB` to form the effective address. In the register-indexed format, `RegX` is added to `RegB` to form the effective address. The computed value is then stored in `RegR`.

Operation

```
RegR <- addOfs32( RegB, signExtend( ofs, 13 ) << dw );
RegR <- addOfs32( RegB, ( RegX << dw ));
```

Exceptions

None.

Notes

None

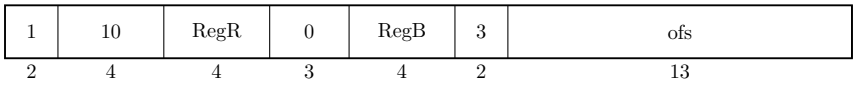
LDR Load Register Reserved

Syntax

`LDR RegR, SignedImmed13(RegB)`

Format

The `LDR` instruction uses the following instruction format:



Description

The `LDR` instruction loads a value from memory into a general-purpose register and marks the effective address as "reserved" for a subsequent conditional store. The instruction is only the first step of an atomic read-modify-write pair. The subsequent `STC` (store conditional) instruction will succeed only if the reservation is still valid.

The signed immediate offset is shifted right by 3 and added to `RegB` to form the effective address.

Operation

```
RegR <- memLoad( memOperandImmIndexed( instr ));
markAddressReserved( memOperandImmIndexed( instr ));
```

Exceptions

- Memory Access Violation Trap
- Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

This instruction is part of a "load-reserved / store-conditional" pair for atomic memory updates and often used for implementing locks or atomic counters. Reservation may be cleared by other stores to the same address by another processor. See the appendix for more information.

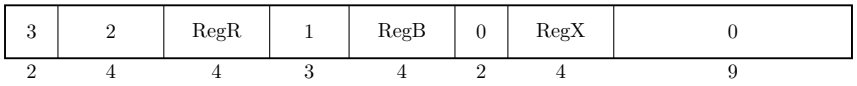
LPA Load Physical Address

Syntax

```
LPA[.D/W/H/B] RegR, RegX( RegB )
```

Format

The `LD` instruction uses the following instruction formats:



Description

The `LPA` (Load Physical Address) instruction computes the physical address corresponding to the given virtual address and loads it into `RegR`. The virtual address is formed by adding `RegX` to `RegB`. If there is no translation, a value of zero is returned.

The physical address, if found, is returned from information in the TLB. No memory access is performed. If there is a separate instruction and data TLB, the data TLB is used. For direct mapped regions the virtual address is the physical address.

Operation

```
RegR <- tanslateAdr( addOfs32( RegB, RegX ));
```

Exceptions

- Alignment Trap
- Privileged instruction trap.
- Non access data TLB trap.

Notes

A return value zero is ambiguous in that it means a non-existent translation or a mapping to page zero.

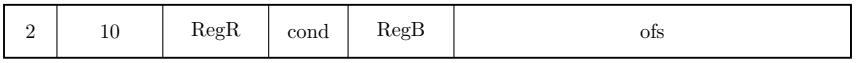
MBR - Move and Branch

Syntax

```
MBR.<cond> RegR, RegB, target
```

Format

The MBR instruction uses the following instruction format:



Description

The `MBR` instruction copies the contents of `RegB` into `RegR`. If the condition specified in the `cond` field evaluates true for the new value in `RegR`, execution continues at the current instruction address plus the sign-extended offset. Otherwise, execution resumes with the next instruction.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == 0`
1	LT	`RegR < 0`
2	GT	`RegR > 0`
3	EV	`RegR.[0] == 0`
4	NE	`RegR != 0`
5	GE	`RegR >= 0`
6	LE	`RegR <= 0`
7	OD	`RegR.[0] != 0`

Operation

```
RegR <- RegB;
if ( cond( RegR )) {

    PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS],
                              signExtend( ofs ) << 2 );
}
else PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], 4 );
```

Exceptions

None.

Notes

None.

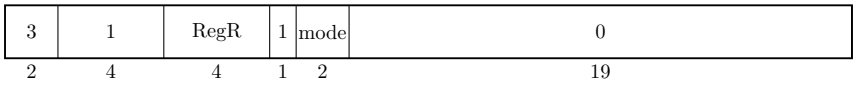
MFIA - Move from Instruction Address

Syntax

`MFIA [.R/L/A]RegR`

Format

The `MFIA` instruction uses the register instruction format.



Description

The `MFIA` instruction copies the current instruction address to register `RegR`.The mode field allows to select a portion of the instruction address.

- `.A` (mode = 0) copies IA to `RegR`. This is the default operation.
- `.L` (mode = 1) Deposits 20 bits of `val` into bits [31..12] of `RegR`.
- `.S` (mode = 2) Deposits 20 bits of `val` into bits [51..32] of `RegR`.

Operation

```
RegR <- PSW.[IA]                          // .A mode (0)
RegR <- (( PSW.[IA] & 0xFFFFF ) << 12 )   // .L mode (1)
RegR <- (( PSW.[IA] & 0xFFFFF ) << 32 )   // .S mode (2)
```

Exceptions

None.

Notes

It would be beneficial to have the possibility to just a subject of the instruction address. How about options to get the segment, the page number in the segment and the status bits. It is very similar to LDIL fields.

??? unclear operation think about what is needed .....

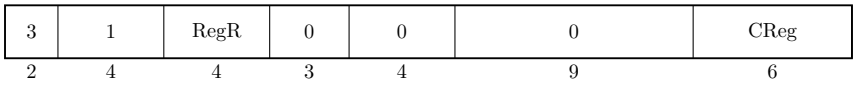
MFCR Move From Control Register

Syntax

```
MFCR RegR, CReg
```

Format

The `MFCR` instruction uses the control-register instruction format.



3	1	RegR	0	0	0	CReg
2	4	4	3	4	9	6

Description

The `MFCR` instruction copies the contents of control register `CReg` to general register `RegR`. Some control registers are privileged. Attempting to read them in user mode results in a privilege exception.

Operation

```
RegR <- CReg;
```

Exceptions

- Privileged Operation Trap for some control registers.

Notes

None

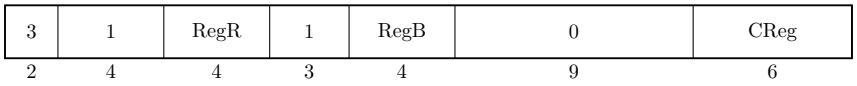
MTCR Move To Control Register

Syntax

`MTCR CReg, RegR`

Format

The `MTCR` instruction uses the control-register instruction format.



3	1	RegR	1	RegB	0	CReg
2	4	4	3	4	9	6

Description

The `MTCR` instruction copies the contents of the control register `CReg` to RegR and then the general register `RegB` to control register `CReg`. Some control registers are privileged. Attempting to write them in user mode results in a privilege exception.

Operation

```
RegR <- CReg;
CReg <- RegB;
```

Exceptions

Privilege operation trap for some control registers.

Notes

None

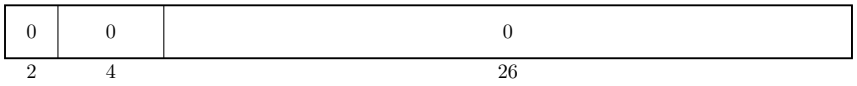
NOP No Operation

Syntax

`NOP`

Format

The `NOP` instruction uses the following format:



Description

The `NOP` (No Operation) instruction does nothing. It is typically used for timing, alignment, or to occupy a pipeline slot.

Operation

```
noOperation( );
```

Exceptions

None.

Notes

None.

OR Boolean Operation

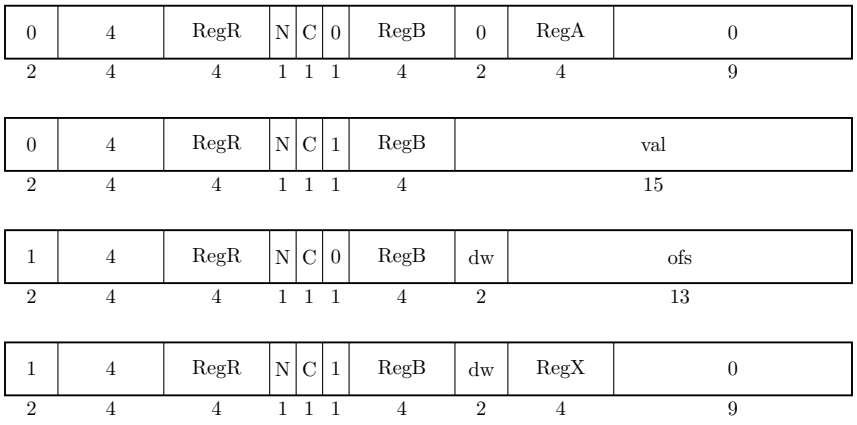
Syntax

```
OR[.C/N] RegR, RegB, RegA
OR[.C/N] RegR, RegB, val

OR[.C/N/D/W/H/B] RegR, ofs( RegB )
OR[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The OR instruction uses the following instruction formats:



Description

The `OR` instruction performs a bitwise OR operation on two 64-bit operands and stores the result in the target register `RegR`. The `AND` instruction performs a bitwise OR operation on two 64-bit operands and stores the result in the target register `RegR`. The second operand, `RegB` or the loaded content, can be negated by setting the "C" bit, and the result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-reference related trap.

Operation

```
tmp <- RegA;                              (S9)
tmp <- immOperand( instr );               (S15)
tmp <- memOperandImmIndexed( instr );     (S13)
tmp <- memOpwerandRegIndexed( instr );    (S9 )

if ( instr.[C] ) tmp <- not( tmp );
RegR <- RegB | tmp;
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

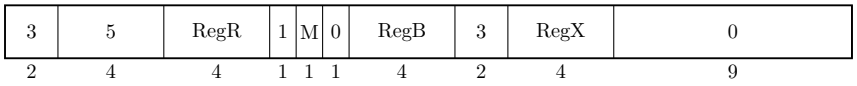
PxCA - Purge from Cache

Syntax

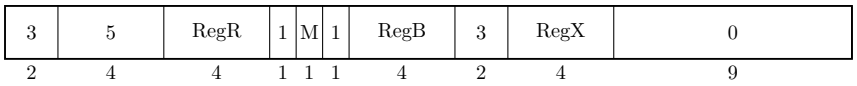
```
PICA RegR, RegB
PDCA RegR, RegB
```

Format

The `PICA` instruction uses the following instruction format:



The `PDCA` instruction uses the standard indexed instruction format.



Description

The `PxCA` instructions purge a cache line, selected by the effective address in `RegB`. This can be used by privileged code for cache management. Unlike `FxCA`, a purge operation may also invalidate dirty lines without writing them back.

Operation

```
RegR <- purgeFromCache( addOfs32( RegB, RegX );
```

Exceptions

None.

Notes

Typically only allowed in supervisor mode.

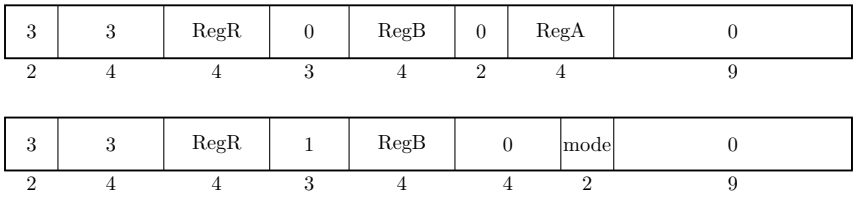
PRB Probe Virtual Address

Syntax

```
PRB RegR, RegB, RegA
PRB RegR, RegB, mode
```

Format

The `PRB` instruction uses both the following instruction format:



Description

The `PRB` instruction probes the Translation Lookaside Buffer (TLB) to determine whether the effective address in `RegB` has a valid mapping. The result of the probe is returned in `RegR`. A value of `0` indicates that no mapping exists, while a `1` indicates a valid mapping.

In the immediate instruction format the access mode is encoded directly in the instruction, otherwise in `RegA`.

The mode specifies the type of access being probed:

Value	Access type
0	`Read access`
1	`Write access`
2	`Execute access`
3	`Undefined`

Operation

```
RegR <- probeVirtualAddress(RegB, RegA.[62..63]);  // Reg
RegR <- probeVirtualAddress(RegB, instr.[mode] );  // Mode
```

Exceptions

- Non-access TLB Trap
- Privileged Operation Trap

Notes

Typically used by the operating system for virtual memory management.

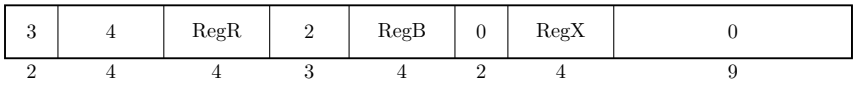
PxTLB Purge TLB Entry

Syntax

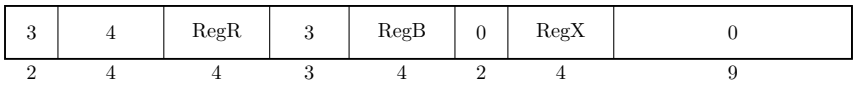
```
PITLB RegR, RegB, RegA
PDTLB RegR, RegB, RegA
```

Format

The `PITLB` instruction uses the following instruction format:



The `PDTLB` instruction uses the following instruction format.



Description

The `PxTLB` instructions purge (remove) an entry from the translation lookaside buffer (TLB). The entry to remove is identified by the effective address formed from `RegB` and `RegX`. If the M bit is set, the base register RegB is updated with the computed effective address.This is privileged instruction.

Operation

```
RegR <- purgeFromTlb( addOfs32( RegB, RegX ));
```

Exceptions

- Privileged operation trap.

Notes

None.

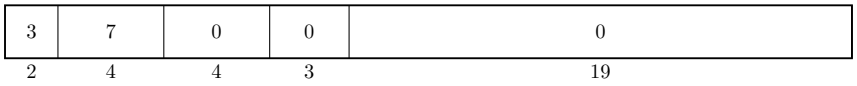
RFI Return from Interrupt

Syntax

```
RFI [ RegR ]
```

Format

The `RFI` instruction uses the branch instruction format.



Description

The `RFI` instruction restores processor state after an interrupt or exception handler has completed. Execution resumes at the program counter saved during the interrupt and the processor state (including privilege level, interrupt enable flags, and other PSR fields) is restored from the saved copy.

Operation

```
PSW <- IPSW;
```

Exceptions

Privileged operation trap.

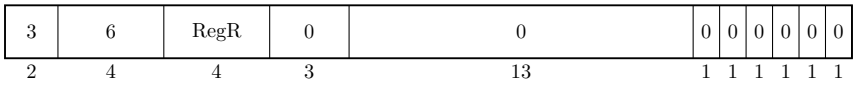
Notes

None.

RSM Reset System Mask

Syntax `RSM RegR`

Format The `RSM` instruction uses the following instruction format:



Description The `RSM` (Reset System Mask) instruction clears bits in the system mask register that are encoded in the instruction. The previous mask values are returned in `RegR`.

Operation

```
SystemMask <- SystemMask & instr.[5:0];
```

Exceptions - Privileged instruction trap.

Notes None.

// ?? what is RegR used for ? return old mask ?

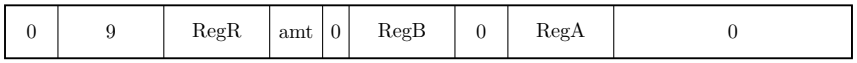
SHLxA Shift Left and Add

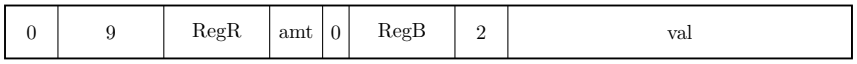
Syntax

```
SHLxA RegR, RegB, RegA
SHLxA RegR, RegB, SignedImmed13
```

Format

The `SHLxA` instruction uses the following instruction formats:





Description

The `SHLxA` instruction shifts the contents of `RegB` left by the amount specified in the `amt` field. In register format, `RegA` is added to the shifted value. In immediate format the sign-extended `val` field is added instead. The resulting value is stored in `RegR`.

Operation

```
RegR <- ( RegB << instr.[amt] ) + RegA;                 (Register
     )
RegR <- ( RegB << instr.[amt] ) + immOperand( instr );  (
    Immediate)
```

Exceptions

Integer Overflow Trap

Notes

None

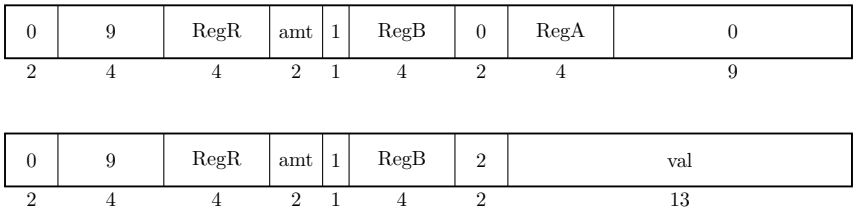
SHRxA - Shift Right and Add

Syntax

```
SHRxA RegR, RegB, RegA
SHRxA RegR, RegB, val
```

Format

The `SHRxA` instruction uses the following instruction formats:



Description

The `SHRxA` instruction shifts the contents of `RegB` right by the amount specified in the `amt` field. In register format, `RegA` is added to the shifted value. In immediate format the sign-extended `val` field is added instead. The resulting value is stored in `RegR`.

Operation

```
RegR <- ( RegB >> instr.[amt] ) + RegA;                 (Register
     )
RegR <- ( RegB >> instr.[amt] ) + immOperand( instr );  (
    Immediate)
```

Exceptions

Integer Overflow Trap

Notes

None

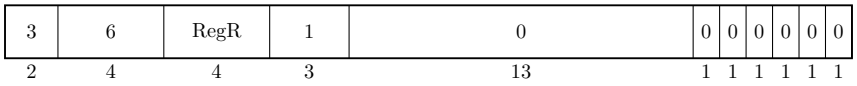
SSM Set System Mask

Syntax

`SSM RegR, mask`

Format

The `SSM` instruction uses the system instruction format.



Description

The `SSM` (Set System Mask) instruction updates the system mask register from the bits encoded in the instruction. The previous mask is returned in `RegR`.

Operation

```
SystemMask <- SystemMask | instr.[5:0];
```

Exceptions

- Privileged instruction trap.

Notes

None.

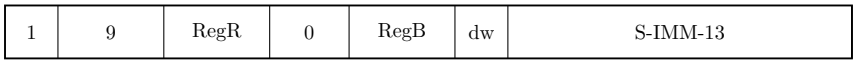
ST Store Register

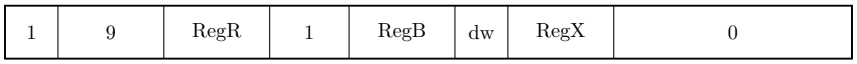
Syntax

```
ST[.D/W/H/B] RegR, SignedImmed13( RegB )
ST[.D/W/H/B] RegR, RegX( RegB )
```

Format

The `ST` instruction uses the indexed and register-indexed instruction formats.





Description

The `ST` instruction stores a value from a general-purpose register into memory.

In the immediate-offset format, the signed immediate offset is shifted right by the `dw` field of the instruction and added to `RegB` to form the effective address. In the register-indexed format, `RegX` is added to `RegB` to form the effective address.

The store address must be aligned to the data width specified. The value stored to memory is stored as is in the data width specified.

Operation

```
memStoreImmIndexed( instr, RegR );
memStoreRegIndexed( instr, RegR );
```

Exceptions

- Data Alignment Trap
- Memory Access Violation Trap
- Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

None.

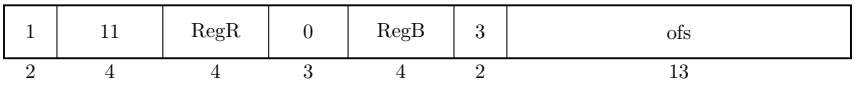
STC Store Register Conditional

Syntax

`STC RegR, SignedImmed13(RegB)`

Format

The `STC` instruction uses the following instruction format:



Description

The `STC` instruction stores a value from a general-purpose register to memory only if the memory address is still reserved by a prior `LDR` instruction. If the reservation is valid, the store succeeds and clears the reservation. If the reservation has been lost (e.g., another processor wrote to the address), the store fails and the memory is unchanged.

Operation

```
if ( isAddressReserved( memOperandImmIndexed( instr ))) {

    memStore( memOperandImmIndexed( instr ), RegR );
    clearReservation( memOperandImmIndexed( instr ));
    RegR <- 1;
}
else RegR <- 0;
```

Exceptions

- Data Alignment Trap
- Memory Access Violation Trap
- Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

This instruction is the second part of a "load-reserved / store-conditional" pair for atomic memory updates and commonly used for implementing spinlocks, mutexes, and atomic counters in multiprocessor systems.

SUB Integer Subtraction

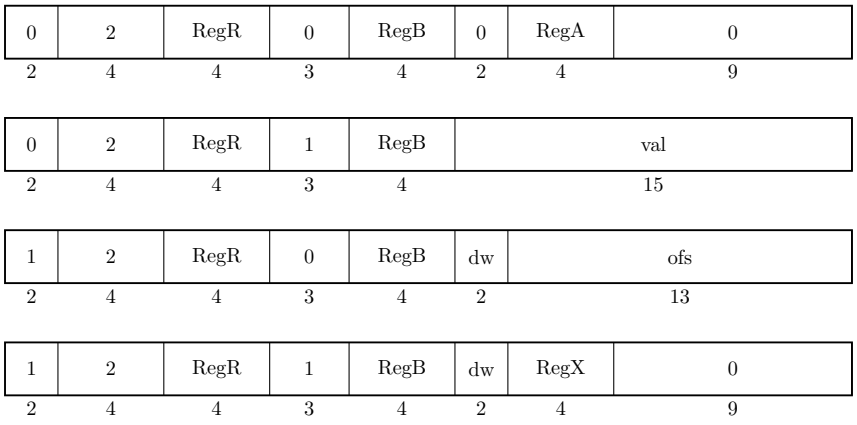
Syntax

```
SUB RegR, RegB, RegA
SUB RegR, RegB, val

SUB[.D/W/H/B] RegR, ofs ( RegB )
SUB[.D/W/H/B] RegR, RegX ( RegB )
```

Format

The SUB instruction uses the following instruction formats:



Description

The `SUB` instruction subtracts two signed 64-bit operands and stores the result in the target register `RegR`. This instruction supports the immediate, register, and both memory-reference formats. An arithmetic overflow triggers a numeric overflow trap, and indexed instruction formats may cause a memory-referencerelated trap.

Operation

```
RegR <- RegB - RegA;                              (S9 )
RegR <- RegB - immOperand( instr );               (S15)
RegR <- RegR - memOperandImmIndexed( instr );     (S13)
RegR <- RegR - memOpwerandRegIndexed( instr );    (S9 )
```

Exceptions

Integer Overflow Trap
Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

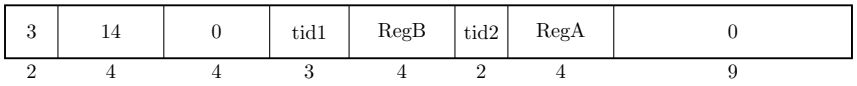
TRAP Trap Instruction

Syntax

`TRAP tid, regB, regA`

Format

The `TRAP` instruction uses the standard indexed instruction format.



3	14	0	tid1	RegB	tid2	RegA	0
2	4	4	3	4	2	4	9

Description

The `TRAP` instruction generates a synchronous trap to handle exceptional conditions. The `tid1` and `tid2` are concatenated to form the trap Id. The instruction may be used to invoke operating system routines or exception handlers.

Operation

```
raiseTrap( concat( tid1, tid2 ), RegB, RegA );
```

Exceptions

The trap raised. See the appendix for valid trap codes.

Notes

None

XOR Boolean Operation

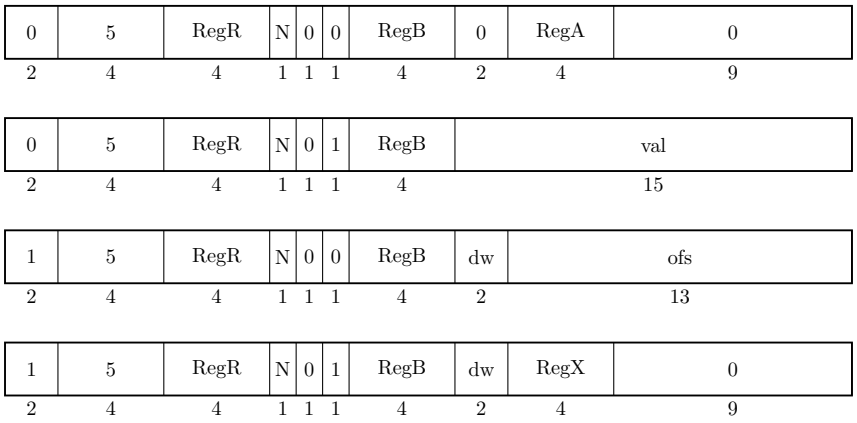
Syntax

```
XOR[.N] RegR, RegB, RegA
XOR[.N] RegR, RegB, val

XOR[.N/D/W/H/B] RegR, ofs( RegB )
XOR[.N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The XOR instruction uses the register, the immediate-19, the indexed and the register indexed instruction formats.



Description

The `XOR` instruction performs a bitwise XOR operation on two 64-bit operands and stores the result in the target register `RegR`. The result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-referencerelated trap.

Operation

```
tmp <- RegA;                              (S9)
tmp <- immOperand( instr );               (S15)
tmp <- memOperandImmIndexed( instr );     (S13)
tmp <- memOpwerandRegIndexed( instr );    (S9 )

RegR <- RegR ^ tmp;
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

Synthetic Instructions

The Twin-64 instruction set provides a rich set of options for individual instructions. Setting a defined option bit in an instruction can enable additional functionality with minimal impact on the overall data path. Similarly, instructions that use general register zero as an argument can take advantage of predefined behaviors.

For better readability and convenience, an assembler can offer *synthetic instructions*, which are higher-level mnemonics generated from the base instruction set. These synthetic instructions simplify programming by providing a more natural or concise representation of commonly used instruction patterns.

Synthetic instructions are also useful when a single instruction cannot fully express the desired operation. For example, loading an immediate value larger than the instruction field allows requires an instruction sequence. An assembler can provide a generic "load immediate" synthetic instruction that automatically emits such an instruction sequence depending on the size of the value. Similarly, if the offset to a base register is too small to reach a desired data item, the assembler can transparently emit an additional 'ADDIL' instruction to compute the correct address.

Immediate Operations

OpCode	**Sequence**	**Comment**
LDI Rn, val	LDO Rn, val(R0)	load an immediate value in the range of $\pm 2^{12}$
LDI Rn, val	LDIL Rn, L%val ADDIL Rn, R%val	load an immediate value in the range of $\pm 2^{31}$

LDI Rn, val	LDIL Rn, L%val
	ADDIL Rn, R%val
	LDIL.M Rn, M%val
		load an immediate value in the range of $\pm 2^{51}$

LDI Rn, val	LDIL Rn, L%val
	ADDIL Rn, R%val
	LDI.M Rn, M%val
	LDIL.U Rn, U%val
		load an immediate value in the range of $\pm 2^{63}$

Register Operations

OpCode	Sequence	Comment
CLR Rn	AND Rn, Rn, 0	clears RegN.
CPY Rn, Rm	OR Rn, Rm, 0	copies RegM to RegN.
COM Rn	XOR Rn, Rn, 1	logically inverts RegN.
NEG Rn	SUB Rn, R0, Rn	negates a value in RegN.

Arithmetic Operations

OpCode	Sequence	Comment
INC Rn, val	ADD Rn, Rn, val	increments Rn by "val".
DEC Rn, val	SUB Rn, Rn, val	decrements Rn by "val".
EXT8 Rn, Rm **EXT16** Rn, Rm **EXT32** Rn, Rm	EXTR.S Rn, Rm, 0, 8 EXTR.S Rn, Rm, 0, 16 EXTR.S Rn, Rm, 0, 32	sign extends a byte in Rn.

Shift and Rotate Operations

OpCode	Sequence	Comment
ASR Rm, Rn, amt	EXTR.S Rn, Rm, amt, 64 - amt	arithmetic shift right by "amt" bits of RegN. The result is stored in RegM.
LSR Rm, Rn, amt	EXTR.S Rn, Rm, amt, 64 - amt	logical shift right by "amt" bits of RegN. The result is stored in RegM.
ASL Rm, Rn, amt	DSR Rm, Rn, R0, amt, 64 - amt	arithmetic shift left by "amt" bits of RegN. The result is stored in RegM.
LSL Rm, Rn, amt	DSR Rm, Rn, R0, amt, 64 - amt	logical shift left by "amt" bits of RegN. The result is stored in RegM.
ROR Rm, Rn, amt	DSR Rm, Rn, Rn, amt	rotate RegN right my amount bits and store in RegM
ROL Rm, Rn, amt	DSR Rm, Rn, Rn, 64 - amt	rotate RegN left my amount bits and store in RegM

Memory Reference Operations

OpCode	Sequence	Comment
LOAD ofs (Rb)	...	...
STOR	...	...

Branch Type Operations

OpCode	Sequence	Comment
BEQ Rn, Rm, target	CBR.EQ Rn, Rm, target	subtract RegM from RegN and branch on condition.
BLT Rn, Rm, target	CBR.LT Rn, Rm, target	
BGT Rn, Rm, target	CBR.GT Rn, Rm, target	
BNE Rn, Rm, target	CBR.NE Rn, Rm, target	
BGE Rn, Rm, target	CBR.GE Rn, Rm, target	
BLE Rn, Rm, target	CBR.LE Rn, Rm, target	
BEQ Rn, target	MBR.EQ Rn, Rn, target	test RegN and branch on condition.
BLT Rn, target	MBR.LT Rn, Rm, target	
BGT Rn, target	MBR.GT Rn, Rm, target	
BEV Rn, target	MBR.EV Rn, Rm, target	
BNE Rn, target	MBR.NE Rn, Rm, target	
BGE Rn, target	MBR.GE Rn, Rm, target	
BLE Rn, target	MBR.LE Rn, Rm, target	
BOD Rn, target	MBR.OD Rn, Rm, target	

Part IV

I/O Subsystem

Input/Output

This chapter presents the Twin-64 I/O architecture and introduces its core concepts related to the processor and system architecture. It does not describe any specific I/O module, nor does it cover operating-system initialization or runtime I/O procedures.

Concepts

Twin-64 uses a memory-mapped I/O architecture. A dedicated portion of the physical address space in address region zero is reserved exclusively for the I/O subsystem. This clean partitioning ensures predictable system behavior during early initialization and simplifies both hardware implementation and software design.

The I/O subsystem is organized around functional entities known as **modules**. Typical modules include processors, memory controllers, and various I/O devices. All modules connect to the **system bus**, which supports up to 64 modules in total. Although real systems often provide fewer physical slots, the architecture itself imposes no such restriction. At system startup, each module is assigned a dedicated 4-KB physical address page. This page defines the base address range to which the module responds and serves as a fixed anchor for its control and status information.

Within each module, functionality is exposed through a collection of **registers**. These registers are accessed using standard load and store instructions, although they are not required to be 64 bits wide. A core set of registers is common across all modules, establishing a uniform interface for essential operations. Each module may define additional device-specific registers to implement its specialized capabilities. The registers within a modules hard physical page are organized into register sets. The first is the architected supervisor register set, while the remaining sets are optional and depend on the particular I/O module.

Many modules also require additional storage to manage their internal **I/O elements**. An I/O elementalso called a **device**is the smallest addressable functional unit within a module. For example, a console terminal module may expose two devices: one for input and one for output.

The system bus spans a 256-KB address space, aligned to a 256-KB boundary, with one 4-KB page reserved for every possible module. In practice, physical constraints often limit the number of installed modules, but the architecture supports growth by allowing multiple modules to reside on a single I/O card, with each card occupying one physical slot on the bus.

Overall, the Twin-64 I/O subsystem emphasizes clarity, modularity, and extensibility. Its hierarchical structurefrom modules to devices, and from hard physical pages to optional soft and broadcast regionssupports both straightforward configurations and more complex, densely populated systems.

I/O Address Space

As shown in the overall physical memory layout, the physical memory address range dedicated to I/O modules is located from `0x00_F000_0000` to `0x00_FFFF_FFFF`. The following figure depicts the I/O memory address space ranges.

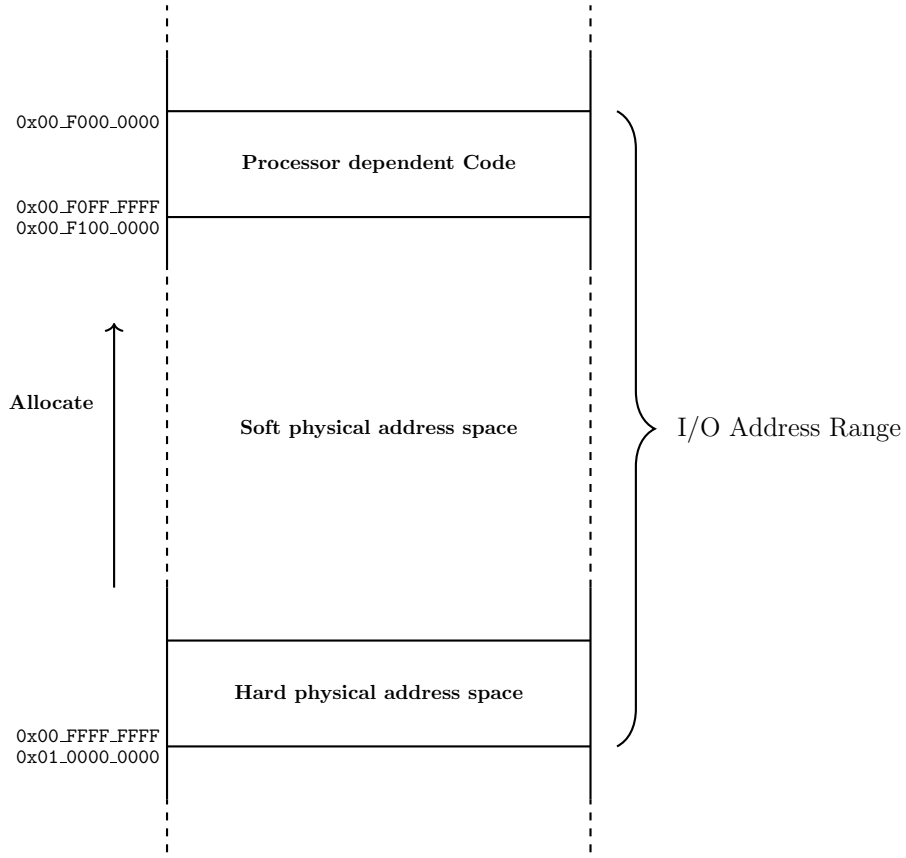


Figure 13: I/O Address Space

The I/O address range is a 256 Mbyte area in the first 4 Gbyte region, i.e. region zero. The area is divided into several ranges. The first range contains the **processor dependent code**. There is a set of library functions to provide low-level primitives for the boot process and the operating system.

At the upper level of the I/O address range, there is the **hard physical address** range and the soft physical address range. It contains a page for each module connected at the system bus level. Up to 63 modules can be allocated. The last module is the broadcast module. At system reset or reboot, there is only the broadcast module. The processor will try to locate all other modules in the physical address range. Finally, each I/O module will then allocate its soft physical address range. Each module therefore listens to three distinct address ranges:

- **Hard physical address range:** This is the modules assigned physical page at system initialization. Its position is determined by the cards physical slot and the modules index on the I/O card. Access to the hard physical page is privileged and reserved for supervisor-level operations.

- **Soft physical address range:** Certain modules require more than a single page of address space to support their full operational state. For such cases, additional memory pages may be dynamically allocated in the soft physical region, providing the necessary flexibility without compromising layout consistency.

- **Broadcast address range:** A write issued through the broadcast module HPA address updates the corresponding register in *every* module on the bus. Broadcast writes are useful for system-wide resets, synchronization operations, or events that must be delivered simultaneously to all modules.

Hard physical address range

The **hard physical address** range contains an I/O page for each I/O module installed. There is room for 63 I/O modules in the system bus. Each module provides a set of register sets in that page. A registers set has 32 registers. A register is a 64-bit word. The first 32 registers are the supervisory registers. This set is architected and must be implemented by each module. It contains the I/O module description and the IODC code entry point table. The remainder of the page contains the I/O ROM code.

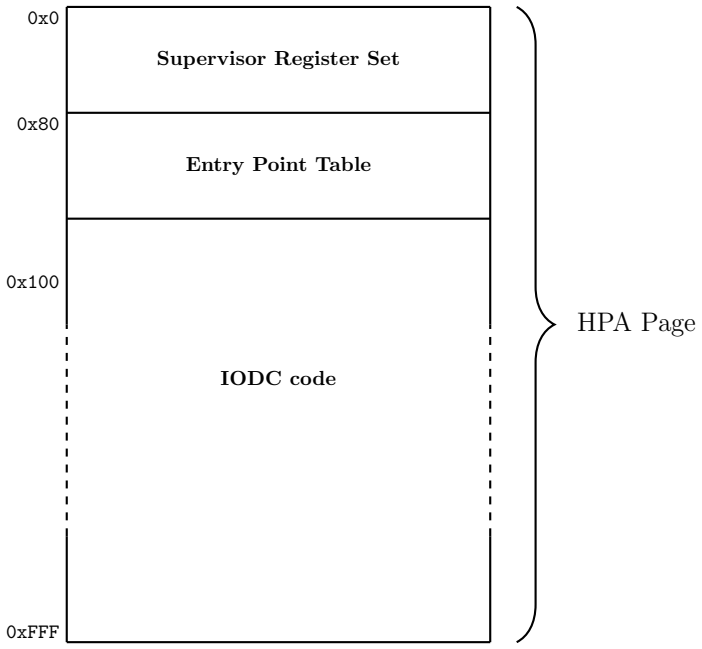


Figure 14: Hard physical Address Page

??? key decision to make: is there only ROM code in the HPA ? then where is the SPA address to find the module data... ?

??? a module could have an SPA address register.

??? all other stuff is then stored in the I/O element, which is in SPA space.

??? in here we also have the IODC code for the functional entry points specific to the module.
??? need to describe in the supervisor register set where this is located in the HPA page.

??? a module still needs some mutable state at the module level:

IO_COMMAND for resetting

IO_ADDRESS (read from HAPOA page)

IO_DATA (data read)

IO_STATUS for module status / health

IO_SPA for finding the SPA address range

??? immutable

IO_MODULE_TYPE

IO_MODULE_ID

IO_MODULE_VERSION

IO_MODULE_CAPABILITIES

IO_ENTRY_POINT_TAB_OFS

IO_ENTRY_POINT_TAB_LEN

Broadcast address range

I/O module 63, i.e. the last page in the HPA address range, is the broadcast module. It is actually not a module and contains no actual functions. Writes issued to broadcastable registers using this module number are not directed to a single module. Instead, the write operation is applied simultaneously to all I/O modules on the system bus. This mechanism is typically used for system-wide commands such as resets, synchronization events, or configuration updates that must reach every module at once.

At reset time, all modules only listen to their broadcastable registers. Only after system bus setup and module discovery, is the HPA address range available.

Soft physical address range

In many cases, a module requires more addressable registers or data structures than can be accommodated within its hard physical address page. The **soft physical address** range provides a flexible region where additional space may be allocated for an I/O modules extended functionality. From a software perspective, the allocated area is termed **I/O Element**. However, the meaning of register sets in this range is entirely up to the module. A module responds to memory accesses within its configured soft physical address space, which may reside either in the I/O address space or the general memory address space.

For example, a memory controller has a hard physical page in the I/O address space, yet its soft physical region typically encompasses all or part of the systems physical memory. Similarly, a processor may allocate a soft physical address range within the I/O space to store performance counters or diagnostic information that needs to be accessible without interfering with normal memory traffic.

??? direct I/O and DMA use SPA space, up to two SPA spaces, so we can map one page in HPA and one in USER space.

??? Processor has PDC as SPA space, each processor has an area in SPA

??? System Console has no SPA

??? IODC data has info what to configure as SPA for a module

??? a module still needs some mutable state at the module level:

IO_COMMAND for resetting

IO_ADDRESS (read from HAPOA page)

IO_DATA (data read)

IO_STATUS for module status / health

IO_SPA for finding the SPA address range

??? immutable

IO_MODULE_TYPE

IO_MODULE_ID

IO_MODULE_VERSION

IO_MODULE_CAPABILITIES

IO_ENTRY_POINT_TAB_OFS

IO_ENTRY_POINT_TAB_LEN

Supervisor Register Set

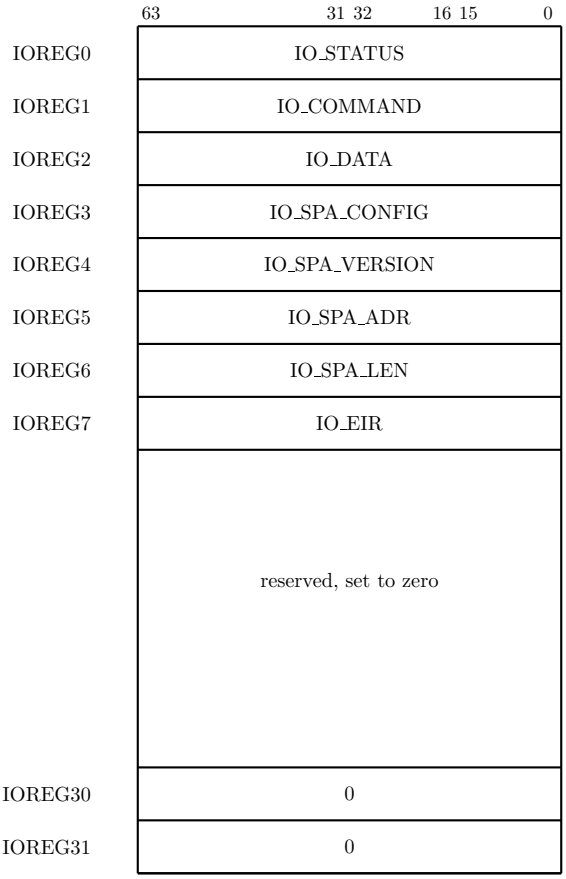


Figure 15: I/O Module Supervisory Registers

??? need to SPA places ?

I/O Elements

Modules consist of I/O elements, smallest I/O unit of operation.

??? several types of I/O Elements.

??? a two page type, where one page is privileged and in I/O space, another one mapped into user space.

??? also option of packing many smaller I/O elements in one page.

Interrupt Processing

what do we need to store for interrupt processing in the registers ???

A processor has two control registers. The external interrupt register and the external interrupt mask register.

Both are ANDed and checked after each instruction execution.

A module can set a bit in the external interrupt request register. There are 64 interrupt groups in the EIRR control register.

When not masked by the EIMR register, the interrupt is handled.

Each module has a EIREQ register. This register contains two fields. The first is the target processor HPA page, the second is the bit position to set in the EIRR register. This is normally set up at system initialization time, but can also be set for each I/O request.

I/O dependent Code

??? what it is

??? how to access

Processor dependent Code

??? should it be in the I/O chapter ?

Design Rationale

??? simple but flexible design

??? closely modeled after PA-RISC, broadcast is simplified

??? key requirement, is that all modules have a description on board in a ROM.

??? PDC lives at the system level, literally as a ROM on the board ?

Part V

Runtime Environment

Runtime Overview

No CPU architecture with an idea of a runtime environment. Although the instruction set can be used to implement many models of runtime environments, there are common principles that exploit the CPU architecture. In fact, several CPU concepts were selected with a certain runtime already in mind. This chapter will present the Twin-64 runtime environment and describe the high level system model, the register usage convention, the execution model and calling convention.

System Environment

The following figure depicts a high-level overview of the overall software environment for the Twin-64 system. At the center of the execution model is the execution thread, referred to as a **task**. A task consists of the currently running execution object together with its private task-local data area, including its own stack and runtime context. All tasks of a job share a common job data area. At the highest level is the **system**, which contains the global data area for the system.

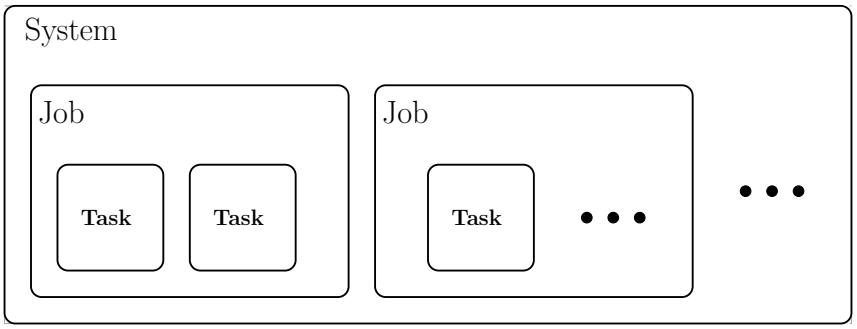


Figure 16: System Environment

All tasks belonging to the same job share a common job data area, which is used to store information and resources that must persist for the lifetime of the job and be visible across all of its tasks. A job may represent an interactive session, such as a terminal session, or it may be a non-interactive batch operation.

The system level contains the global data area, system libraries, and all services provided by what is traditionally called the operating system. The system is responsible for creating jobs, managing their lifetime, and scheduling the tasks within them.

The number of active tasks at any moment typically corresponds to the number of available processors in the system, allowing the system to take full advantage of available parallelism. While each task maintains its own private data and execution state, tasks within a single job share access to the job data area. All tasks, regardless of job, also have access to system-level services and global resources.

Runtime Environment

The processor is a general-purpose CPU. The runtime model presented here represents one possible implementation strategy, chosen to support the calling conventions and execution model that will be described in later sections. The following figure depicts a high-level overview of the runtime environment.

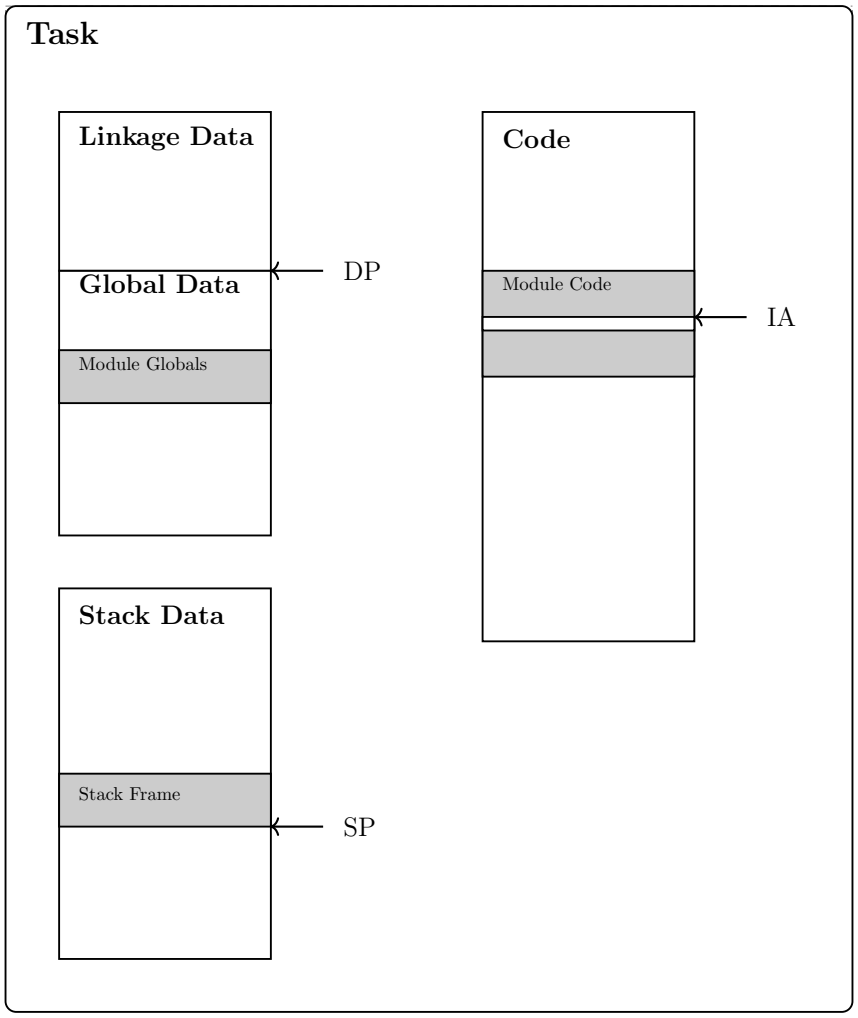


Figure 17: Runtime Environment

When a compiler or assembler generates code, the fundamental unit of work is the compilation unit. A compilation unit corresponds to a single invocation of the compiler or assembler. Although syntax varies by language, the source text is organized into modules. A module is an entity that contains constant declarations, type definitions, module-level global variables, and functions.

The tasks code region contains the program text of all modules used by the task. Once loaded, this region is immutable. The currently executing instruction is identified by the instruction address (IA).

Each task also has a global data region. This task-level global data is formed by aggregating the global data sections of all modules referenced by the task. It is addressed using the data pointer (DP) plus an offset. The data region includes the tasks stack, which holds stack frames

created by function calls. The active frame is located via the stack pointer (SP), and items within the stack are accessed relative to SP.

The data region further contains the linkage table, a structure that provides the information needed to access functions in external libraries. Details of this mechanism are described in the section on linkage tables.

In this runtime design, the stack grows upward: new frames occupy increasing addresses, and older frame contents are typically referenced using expressions of the form SP - ¡ofs¿. Each stack frame includes a frame marker containing the previous SP and the return link (RL), enabling structured returns and proper unwinding of frames.

Note.

Register Usage

The CPU features general registers and control registers. All general registers can do computation as well as address computation. The runtime architecture builds on the general register file and assigns specific meaning to each register.

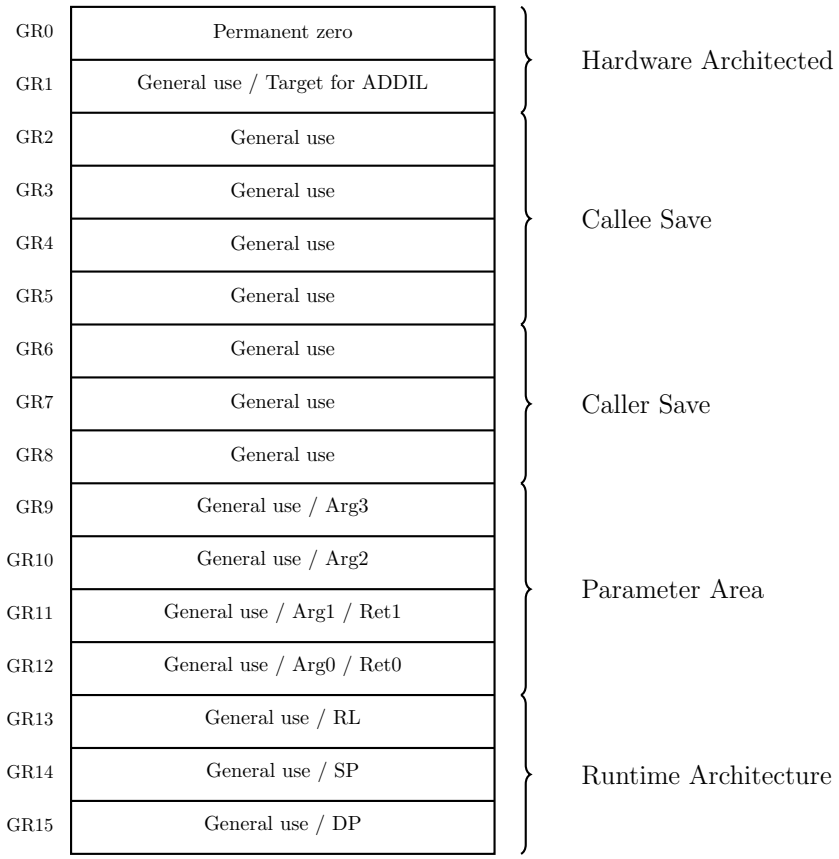


Figure 18: General Register Usage

Registers `R0` and `R1` are hardware-architected and have fixed, predefined roles within the processor. The remaining registers, `R2` through `R15`, form the general-purpose register file, whose usage is defined by the runtime environment.

Registers `R2` through `R5` form the callee-save register set. A callee-save register is one whose value must be preserved by the called function: if the function uses such a register, it must save its original contents upon entry and restore them before returning. This ensures that long-lived values in the caller remain intact across function calls.

Registers `R6` and `R8` are designated as caller-save registers. A caller-save register may be overwritten by the called function; therefore, the caller is responsible for saving the registers value before making a call if it must be preserved.

Argument passing is handled through registers `R9` to `R12`, known as the parameter registers. When a function is invoked, its first four arguments are placed into these registers. Registers `R11` and `R12` also serve as the return-value registers, allowing a function to return up to two values without using memory.

Control flow is supported by register `R13`, the return-link register (RL). During a function call, the branch instructions store the return address in `R13`, allowing the function to return control to its caller.

Register `R14` serves as the stack-frame pointer (SP). It marks the base of the current stack frame, which contains local variables, spilled registers, and temporaries, and supports predictable stack-based data management within the tasks runtime environment.

Finally, register `R15` is assigned as the data pointer (DP). It serves as the base reference for accessing the tasks global data region.

Stack Layout

Most modern programming languages rely on a **stack** to manage function calls, local variables, and return information. This stack is organized into **stack frames**, each representing the state of an active function invocation. When a function is called, a new frame is pushed onto the stack; when it returns, that frame is popped. This structured approach allows programs to keep track of nested calls efficiently and ensures that each function has its own isolated workspace during execution.

Stack Frame

Stack frames are aligned on a 16-byte boundary. A stack frame consists of four main areas, depicted in the following figure.

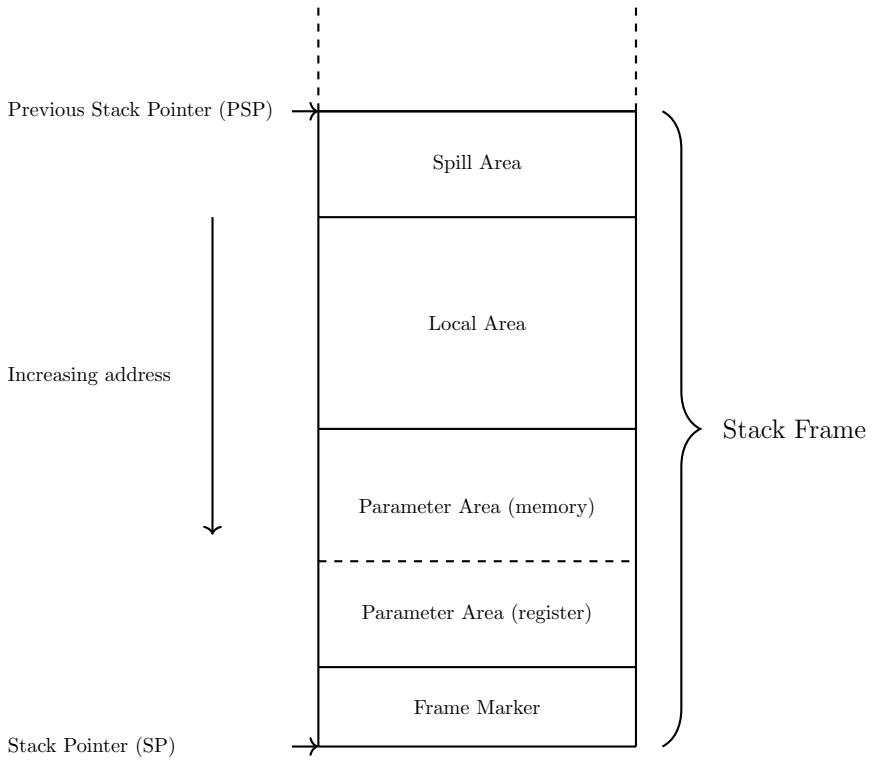


Figure 19: Stack Frame

The **stack frame marker** holds a pointer to the previous stack frame along with the return link address, allowing the call chain to be restored during function returns.

The **parameter area** is used for passing arguments to the called function and for returning data. Although the first four parameters are passed in registers, space for them must still be reserved in memory. Any parameters beyond the first four are placed directly in this area.

The **local area** stores the functions local variables. This is the region the function uses for data that must persist throughout its execution.

Finally, the **spill area** is used to save registers whose values must be preserved and restored before returning to the caller.

Stack Frame Addressing

All data items within a stack frame are accessed **relative to the current stack pointer (SP)**. The architecture uses an `SP - offset` addressing scheme, where each region of the stack frame, i.e. the frame marker, parameter area, local data area, and spill area are located at a fixed negative offset from the SP at function entry. This convention allows the compiler and debugger to compute the location of every stack-resident object from the SP alone, without requiring a dedicated frame pointer. As long as the SP remains stable during the execution of the function, all stack frame items can be reliably referenced through their predefined offsets.

Stack Frame Marker

A stack frame marker consists of two double words. The frame marker is pointed by the current stack pointer register (SP).

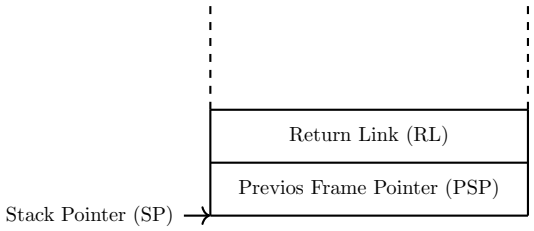


Figure 20: Stack Frame Marker

The first word holds the **address of the previous stack frame**. This field does not necessarily store the previous frame marker address, as the frame size can often be inferred from the instructions that created the current frame together with symbolic debugging information in the program image.

The second word contains the **return link**, which is the address immediately following the branch instruction in the calling function. If the called function is a leaf function, there is no need to save the return link in memory.

Parameter Area

A stack frame includes a **parameter area** large enough to hold the largest argument list passed from the calling function to the callee. Although the first four parameters are normally passed in registers, corresponding space is still reserved in the frame to provide a uniform layout and to support cases where the callee needs to spill register arguments to memory. Any additional

parameters beyond the first four are passed directly in the memory locations immediately below this register-parameter area. If a function neither receives parameters nor returns data, the parameter area is omitted from its stack frame, avoiding unnecessary stack usage.

Local Data Area

Immediately below the parameter area lies the **local data area**, which holds all automatic (local) variables of the function. These variables exist only for the duration of the function call and are allocated anew for each invocation. The compiler determines the size and placement of each local variable within this area, taking into account alignment requirements and optimization opportunities. If a function declares no local variables, this region has size zero and is omitted from the final stack frame layout.

Spill Area

The **spill area** contains any registers that the function must temporarily save in order to preserve the calling convention. Registers that must remain unchanged across function calls are saved ("spilled") into this area upon entry and restored before returning to the caller. The compiler allocates spill slots only when necessary; if a function does not need to save any registers, the spill area is not present in its stack frame. This selective allocation helps keep stack usage minimal while maintaining correct program behavior.

Calling Convention

The previous chapters introduced the register-usage conventions and the overall stack layout defined by the runtime architecture. Building on that foundation, this chapter describes the mechanisms used to invoke routines and the conventions that govern control transfer, parameter passing, register preservation, and stack-frame management.

The discussion begins with the simplest form of call: the *local call*, which transfers control between routines within the same module. Local calls illustrate the fundamental steps of the calling convention without the additional considerations required for inter-module calls.

A routine call proceeds through several well-defined stages. First, the caller evaluates all argument expressions and places the resulting values into the parameter area, either in registers or on the stack as required. After preparing the parameters, the caller saves any registers whose values must remain intact across the call. These caller-saved registers must be preserved by the caller before branching to another routine.

The callee then begins execution at its entry point. It accesses the incoming parameters in the established layout of the parameter area, allocates a stack frame, records the return link, and saves any callee-saved registers it intends to overwrite. This setup ensures that caller and callee operate independently while maintaining a consistent runtime state.

When the callee completes its work, it restores all callee-saved registers, retrieves the return link from the frame marker, deallocates its stack frame, and transfers control back to the caller. This division of responsibility allows efficient cooperation between routines and avoids unnecessary state preservation.

As described earlier, the general-purpose register set is partitioned into caller-save and callee-save regions. A compiler selects registers from these regions as needed, balancing the work of saving and restoring state between the caller and the callee. When possible, redundant saves and restores are eliminated. The runtime architecture distinguishes between two basic categories of calls: *local calls*, which remain within a single module, and *external calls*, which cross module boundaries and require additional handling. The following sections describe these types in detail.

Local Call

A local call is the most common form of procedure call. The following code snippet illustrates such a call. The caller loads the arguments, saves any registers the callee might modify, and branches to the callee. After the callee returns, the caller restores its saved registers and continues execution.

```
; calling procedure

            ...                 ; load arguments
            ...                 ; save caller saves
            BL target, RL       ; branch to "target", save return
            ...                 ; restore caller saves
```

The callee begins by saving the return link, allocating its stack frame, and saving any registers that must remain unchanged across the call. When the routine completes, it restores the return address, restores callee-saved registers, deallocates the stack frame, and branches back to the caller.

```
; called procedure

target:     ST RL, -16(SP)      ; save RL
            LDO SP, size(SP)    ; allocate stack frame
            ...                 ; save callee save

            ...                 ; do the work


            ...                 ; restore callee saves
            LDO SP, -size (SP) ; deallocate stack frame
            LD RL, -16(SP)      ; restore RL
            BV (RL)             ; return to caller
```

This sequence shows all steps that may be required for a local call. Depending on the specific call and the nature of the callee, some steps may be omitted. For example, if the caller does not use any caller-saved registers, no saving is needed before the call. Likewise, a routine with no arguments does not need to evaluate arguments.

The callee can also omit steps. A *leaf procedure*, i.e. one that calls no other routines, does not need to save the return address. If it does not use any callee-saved registers, those need not be saved or restored. Finally, if a leaf procedure uses no caller-saved registers, has no local variables, and does not need to preserve temporary registers, it may require no stack frame at all.

The instructions `B`, `BR`, and `BV` are used for local branches. The address computation, whether IA-relative or region-relative, does not modify the region portion of the instruction address. IT is always the region of the currently executing code.

Parameter passing

Procedures may pass arguments to a callee and receive return values. The parameter area is located in the caller's stack frame, below the frame marker.

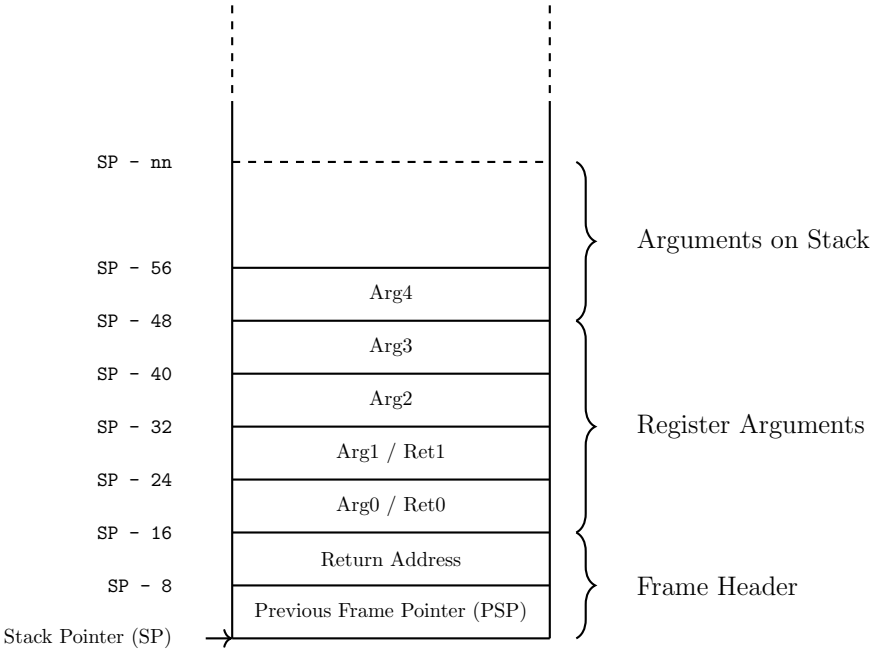


Figure 21: Argument Passing

The first four arguments are passed in registers `ARG0` through `ARG3`. Nevertheless, the caller's stack frame reserves space for them in the parameter area. Any additional parameters are stored on the stack. Upon return, the callee may supply up to two return values in registers `RET0` and `RET1`.

Arguments and return values are addressed relative to the caller's frame pointer. Because the callee may allocate its own stack frame, the size of the callee's frame must be added to the fixed offsets of the arguments in the caller's frame.

External Calls

An external call is one that may cross a region boundary. The `BE` instruction computes the effective address from a general register and an immediate offset encoded in the instruction. Like local branches, this computation produces a 32-bit unsigned address offset. However, unlike local branches, the base register contains the full virtual address, region and offset, not just an offset. The `BE` instruction is used both to call a target in another region and to return from it. The following code snippets shows a basic external call.

```
1
2 ; calling code
3
4             ...                 ; load arguments
5             ...                 ; save caller saves
6             LD R1, <target>)    ; obtain the target address
7             BE R1, RL           ; branch to "target", save return
8             ...                 ; restore caller saves
```

This snippet illustrates only the external branch. The full runtime convention is more elaborate. When calling an external function, the target address and its associated DP value must be obtained. This information is stored in the **linkage table**, which is populated with actual values at load time. This indirection allows compiled code to remain unchanged regardless of where the external function is ultimately located.

Because there is no practical way for the callee to determine its correct DP value in a shared-code model, the caller is responsible for setting DP before branching outside its module. The necessary steps of loading the target address, loading the target DP, and performing the external branch are placed into a small piece of code called a *stub*. A stub is generated at compile time and inserted into the module's code.

A generic external call sequence is shown below:

```
1
2  ;calling code
3
4              ...
5              ...                      ; load arguments
6              ...                      ; save caller saves
7              ...                      ; save DP
8              B stubF1                 ; branch to the stub for "F1"
9              ...                      ; restore DP
10             ...                      ; restore caller saves
11             ...
12
13 ;stubF1
14
15 stubF1:     LD R1, -ofs(DP)          ; load target address of F1
16             LD DP, -ofs + 8(DP)      ; load target DP value
17             BE (R1)                  ; branch to target
```

Before the stub loads the new DP value, the caller must save the current DP. This step is always required, even if the calling function does not use DP. The Twin-64 runtime model performs this work in the caller's code before branching to the stub. This division has several benefits. First, the caller can optimize DP saving. For example, by dedicating a register to hold it so that only restoring DP is necessary after external calls. Second, the stub's sequence of loading the target address and DP value is shared for all calls to the same external function. Finally, the target function is unaware of whether it was reached by a local or external call:

```
1
2  ; called code
3
4  target:     ST RL, -16(SP)       ; save RL
5              LDO SP, size(SP)     ; allocate stack frame of <size>
6              ...                  ; save callee save regs
7
8              ...                  ; do the function work
9
10             ...                  ; restore callee saves
11             LDO SP, -size(SP)    ; deallocate stack frame
12             LD RL, -16(SP)       ; restore RL
13             BE (RL)              ; branch to return location
```

Local calls may use `BV` to return, since the region portion of the address does not change. Exported functions, however, may return either locally or externally, and therefore must use `BE`. It is recommended to simply always use the `BE` instruction.

Linkage Table

The assembler or compiler generates the skeleton of the linkage table. The linkage table resides below the global data area. Entries are addressed using negative DP offsets.

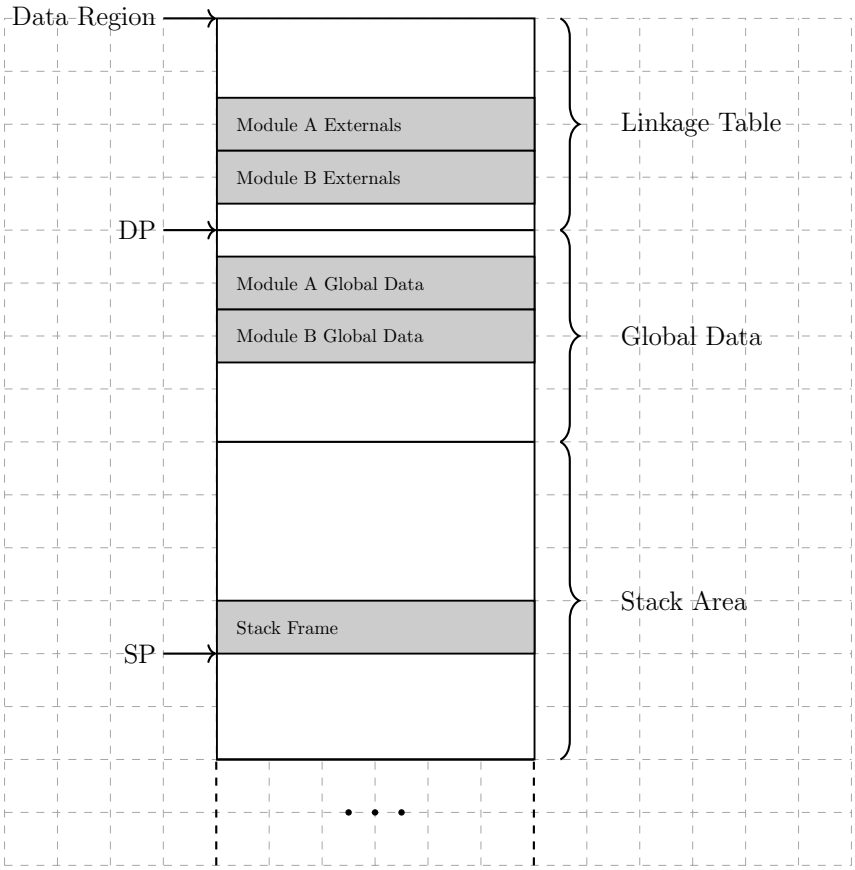
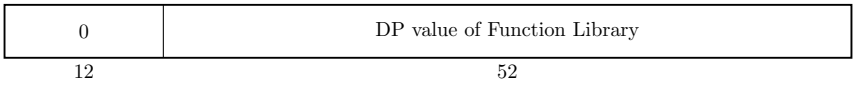


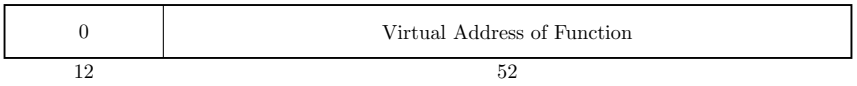
Figure 22: Linkage Table

The figure illustrates two modules within a task's global data region. A module's global data is addressed as DP plus an offset, while linkage-table entries are addressed as DP minus an offset. The compiler constructs the linkage table during compilation. The linkage table contains per external library several entries.

The DP value entry starts the linkage subtable for a particular library.



The function target entry contains the target address of the function.



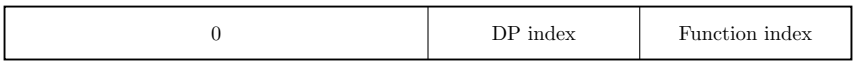
The runtime supports two methods for allocating linkage table entries. In the first method, each external function has its own pair of entries, the DP value followed by the target address. These can be accessed using constant offsets of 0 and 8 bytes, respectively.

However, because the DP register value is identical for all external functions within a given external load module, it only needs to be stored once. In the second method, a single DP entry is placed at the start of the linkage table portion for the external module, followed by the target address entries for all external functions. This approach produces a more compact table.

Function Labels

A function label is a reference to a function which is resolved at calling time. Therefore each function that is accessed via a function label has an entry in the linkage table. The function label itself is then the index into the linkage table.

??? the entry is two pieces: DP index and function index. To be defined...




```
1
2 ; example
3             ...                 ;
4             ...                 ;
```

Privilege Level Changes

```
1
2 ; example
3             ...                 ;
4             ...                 ;
```

Design Rationale

There are many possible ways to organize modules and their external linkages. Classic CISC architectures provided simple call instructions that implicitly performed all the required steps such as address computation, access-rights checking, and memory references needed to resolve external linkages.

RISC designs, in contrast, expose only simple branch and jump instructions and rely on explicit caller-callee code sequences to implement calling conventions. Because compilers require a

unified and predictable calling sequence, the additional work needed for external calls is typically done via import and export stubs.

Twin-64 follows a similar model. External calls use an import stub, which is responsible for locating the target function and obtaining the external librarys DP value before issuing the external branch instruction. The caller saves its own DP value and restores it upon return.

This approach adds two simple instructions at each external call site, but provides a key benefit: it avoids two additional branches, one from the import stub to the callee and one back from the import stub to the caller. It also centralizes DP handling. Because the stub does not manipulate the callers DP value, the caller has full flexibility. It may save DP to memory, move it to another register, or even save it once and reload it after each external call, depending on optimization goals.

Thus an external call requires only one additional branch in the calling sequence, and the code expansion for saving the callers DP is modest. The callee is completely unaware of the callers region, so no export stub is required.

An additional advantage is that the compiler does not need to generate any special code for functions that may be called externally. Only imported functions must be DP-aware. This reduces complexity and places less burden on the compiler designer.

Part VI

Simulator Environment

Twin-64 Simulator

The Twin-64 simulator is an interactive program that runs a configured Twin-64 system. Running the simulator simply means launching the simulator application in a terminal window. During the execution of the configured system, data can be displayed and modified. The simulator features a processor model, system memory and a simulated I/O system. The simulator terminal screen displays system data and allows the monitoring, modifying and tracing of the system execution.

Simulator Environment

The simulator environment is a set of configured modules. A module is a processor, memory or I/O module. The following figure shows a high level view of a configured system.

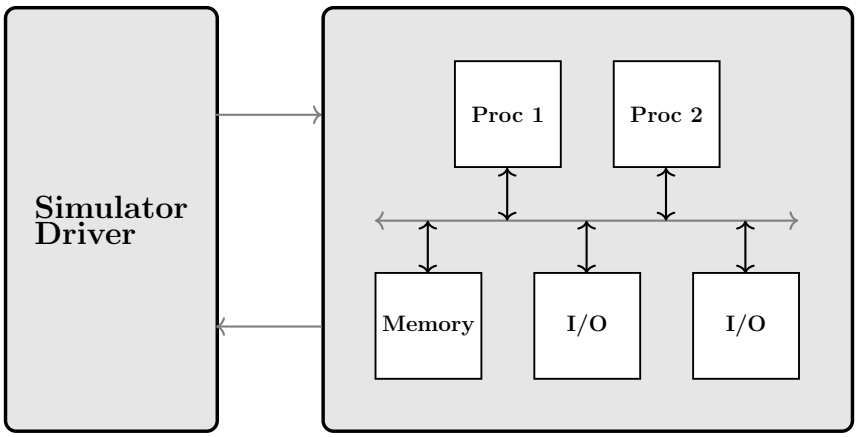


Figure 23: Simulator Environment

The figure above shows a two processor system with a memory module and two I/O modules an a shared **system bus**. The simulator driver issues commands to the simulation environment. Examples are modifying a memory cell content, or single stepping the processors. The simulated environment provides execution data to the driver.

The data is generally shown in **simulator windows**. The main window is the command window. The simulator displays them arranged within a single terminal screen. Each window begins with a banner line shown in inverse video. The banner identifies the window and indicates its current state. The last window on the terminal screen is always the command window.

The simulator normally operates with multiple windows visible, but it can also be placed into **line mode**. In this mode, only the command window is displayed. Commands are entered at the prompt line, and all output is shown sequentially, one line at a time.

Command Window

The **command window** is the primary interface where commands are entered and line-mode information is displayed. This window is always visible, regardless of whether the simulator is operating in windowed mode or line mode. It is always the last window displayed, that is, it appears at the bottom of the terminal screen.

Figure 24: Simulator Main Window

The command window prompts the user to enter a command.

```
(commandNum) ->
```

The command number is an incrementing value. Each command entered is assigned a unique number, which can be used to re-execute or edit previously entered commands. Several commands display the command history, showing recent commands along with their associated command numbers. The `DO` command re-executes the specified command, while the `REDO` command retrieves a command from the history and presents it for editing before execution.

Command lines are case insensitive. Internally all input, except those in quotation marks is upshifted before interpretation.

Expressions

Numeric data entered and displayed is shown in decimal or hexadecimal. A decimal number is a numeric string of the number digits 0 to 9. A hexadecimal number has the prefix "0x" and accepts the digits 0 to 9, A to F. Where the context is clear, a hexadecimal number may drop the "0x" prefix on output. A string is a sequence of characters enclosed in quotation marks.

```
00123456            a decimal number
0xc00f0010          a hexadecimal number
"simulator"         a string
```

Simulator Windows

At the heart of the simulator is the concept of a window to show and modify the simulation environment.

Window Concepts

A window consists of a **window banner line** and a **window body**. The banner line begins with the window identification, which includes the window stack number and the window number. The window number is a unique identifier that can be used to refer to a specific window. Beneath the window banner line is the window body, which consists of several lines of ascii text. Both areas are window types specific and display status information, messages and simulator output.

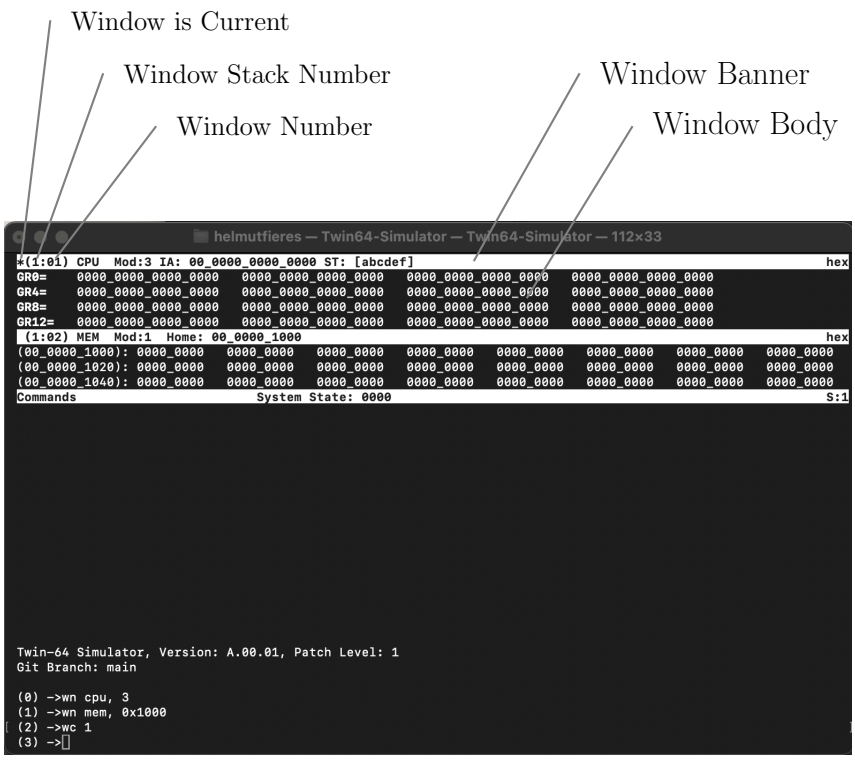


Figure 25: Simulator Windows

Windows can be organized into stacks. This allows related windowsfor example, all windows associated with a single CPUto be grouped together. A stack can be displayed or hidden as a whole. At all times, exactly one window is designated as the current window. The current window is marked with a star (*) symbol in the banner line. If a window command does not explicitly specify a window identifier, it operates on the current window.

The remainder of the banner line is specific to the window type and is described in the following sections of this manual. At the right edge of the banner line, each window displays the radix

format used for presenting its data. The command window banner line displays the currently defined window stacks at the right edge.

Window Stacks

Simulator windows can be organized into **window stacks**. All visible stacks are displayed horizontally with the terminal screen size adjusted accordingly. The following figure shows a two stack example.

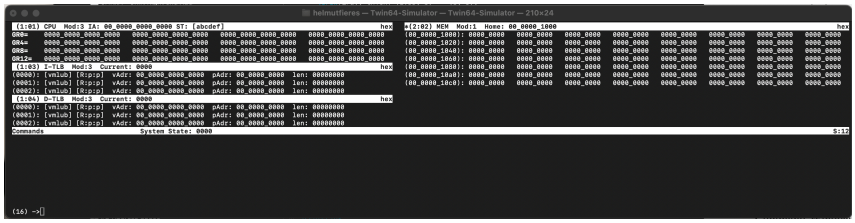


Figure 26: Simulator Window Stacks

Naturally, a monitor screen cannot display more than two or three stacks next to each other. Stacks, like windows, can therefore be disabled. A disabled stack is not shown. The command window has a little status filed in its banner line which shows what stacks have been created.

CPU Window

The CPU window displays the register set of a processor. Several view modes are available, between which the user can toggle. The default view shows the general register set in the window body. The banner line displays the window identifier, the processor module number, the current instruction address, and the CPU status register.

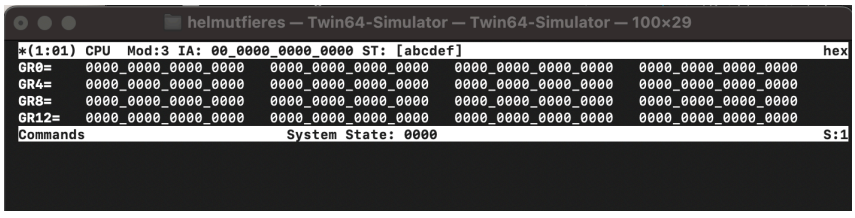


Figure 27: Simulator CPU Window General Registers

In addition to the general registers, control registers that are visible to user-mode programs can be displayed. These registers are shown when the window toggle command is issued.

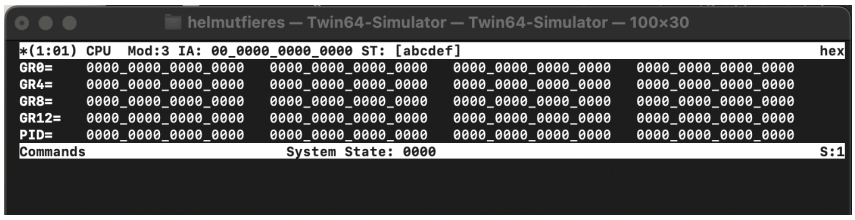


Figure 28: Simulator CPU Window User-Visible Control Registers

Toggling the window once more displays the full set of control registers.

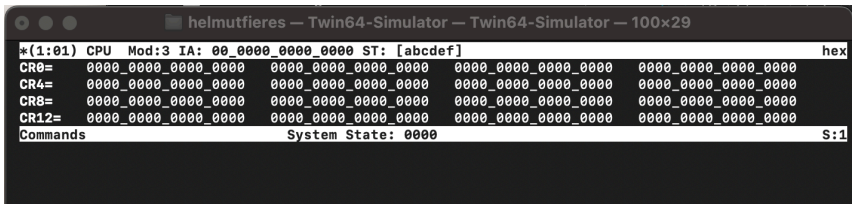


Figure 29: Simulator CPU Window Control Registers

As with all simulator windows, multiple instances of the same window type may be created. In the case of the CPU window, all three view modes can be displayed simultaneously by creating multiple windows and toggling each one to the desired view.

Cache Window

In addition to the CPU core, a processor may include one or more caches. The simulator supports instruction caches, data caches, and optionally a unified cache. All cache-related information is displayed in cache windows.

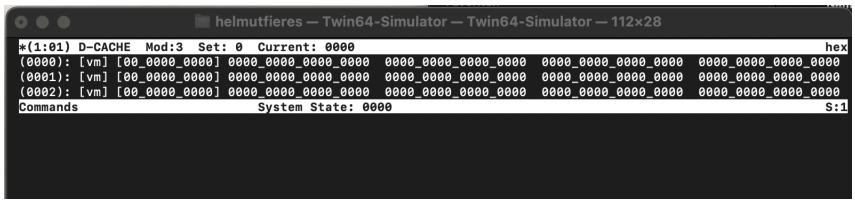


Figure 30: Simulator Cache Window

A cache window displays the cache type, the processor to which it belongs, the currently selected cache set, and the line index at which the window body begins. The window body presents cache contents line by line. Each cache line includes its status bits, tag value, and cache block data.

TLB Window

The TLB (Translation Lookaside Buffer) window displays the contents of the processors address translation cache. The TLB accelerates virtual-to-physical address translation by caching recently used page table entries. The simulator supports instruction TLBs, data TLBs, and optionally a unified TLB. All TLB-related information is presented in TLB windows.

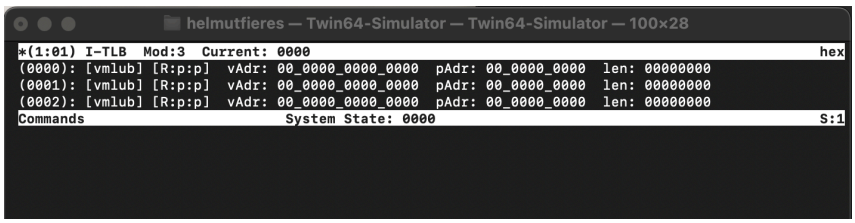


Figure 31: Simulator TLB Window

A TLB window displays the TLB type, the processor to which it belongs, the selected TLB set or entry range, and the index at which the window body begins. The window body shows TLB entries line by line. Each entry includes the virtual page tag, the corresponding physical page number, and the associated status and permission bits.

Memory Window

The memory window displays the contents of physical memory. The banner line contains the window identifier, the memory module number, and the window's home address. This information allows the user to quickly identify the memory region being examined.

The memory window provides several display modes that allow memory data to be viewed in different formats, depending on the debugging task at hand. The window can be toggled between these modes while maintaining the same base address and line alignment.

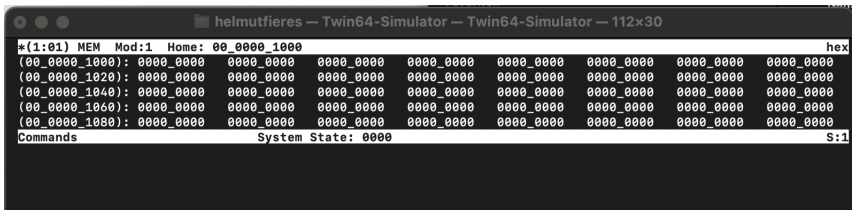


Figure 32: Simulator Memory Window Hexadecimal (32-bit)

In the 32-bit hexadecimal display mode, memory is shown as a sequence of 32-bit words formatted in hexadecimal notation. This view is useful for inspecting instruction streams and word-oriented data structures.

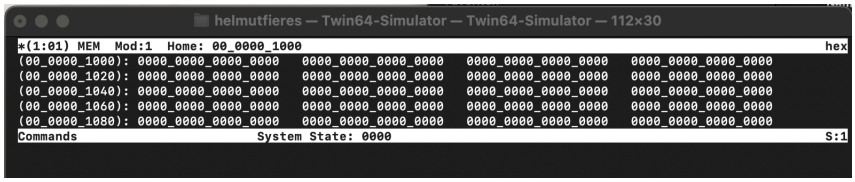


Figure 33: Simulator Memory Window Hexadecimal (64-bit)

The 64-bit hexadecimal display mode presents memory as 64-bit quantities. This format is convenient for examining double-word data values and pointer-sized objects.

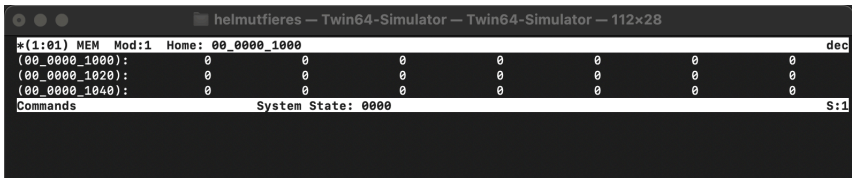


Figure 34: Simulator Memory Window Decimal

In decimal mode, memory contents are displayed as signed decimal values. This view is intended primarily for examining numeric data produced or consumed by applications.

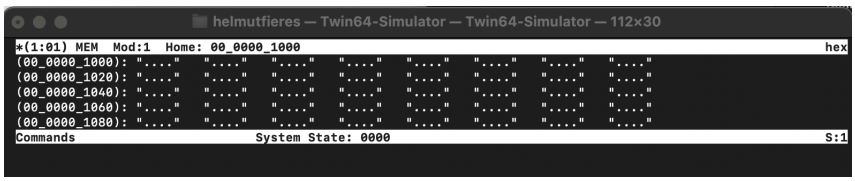


Figure 35: Simulator Memory Window ASCII

The ASCII display mode interprets memory bytes as printable characters. Non-printable bytes are shown using a placeholder representation. This mode is useful for inspecting strings, text buffers, and structured data with embedded character fields.

Code Window

The code window displays the contents of memory in a disassembled form, allowing the user to examine instructions as human-readable assembly code. This window helps in debugging by showing the relationship between machine code in memory and its corresponding assembly instructions.

The banner line contains the window identifier, the memory module number, and the window's home address, allowing the user to quickly identify which memory region is being disassembled.

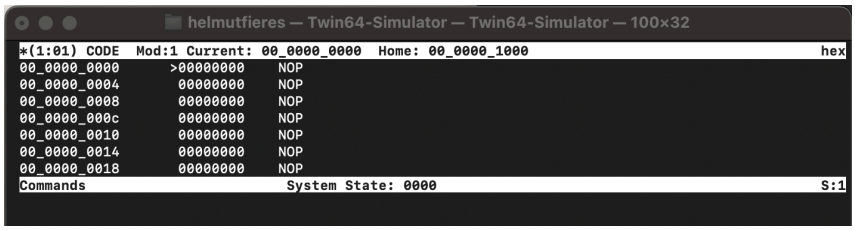


Figure 36: Simulator Memory Window Disassembled Code

Each line in the code window represents a memory address and the instruction stored at that address. Additional information such as instruction operands, addresses, and instruction lengths may also be displayed, depending on the simulator configuration.

Text Window

The text window displays the contents of a text file. The banner line contains the window identifier, the memory or file module number, and the window's home address, allowing the user to quickly identify the source and location of the displayed text.

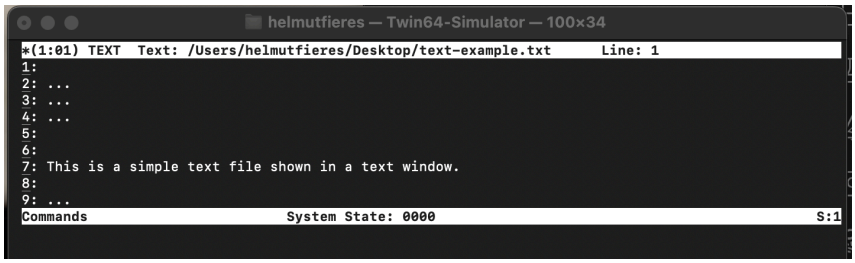


Figure 37: Simulator Text Window

The text window displays the content line by line. It preserves formatting such as line breaks and spaces, making it suitable for inspecting textual data exactly as it would appear in a file or console output.

Simulator Line Commands

This chapter presents the line mode commands. Although they are called line mode commands, these commands are also available when windows are enabled. The following table is summary of the available commands. Like all commands, they are entered in the command window.

Command Summary

Table 12: Simulator Line Commands

Command	Purpose
DA	display absolute memory
DM	display configured Twin-64 modules
DO	execute a command from the command history stack
DW	display windows
ENV	display, add, modify environment variables
EXIT	leave the Twin-64 simulator
FDCA	flush data cache from CPU
FICA	flush instruction cache from CPU
HIST	display command history stack
IDTLB	insert into a data TLB of a CPU
IDTLB	insert into a instruction TLB of a CPU
LF	load ELF file into memory
MA	modify absolute memory
MR	modify a register of a CPU
PDCA	purge from data cache of a CPU
PDTLB	purge from data TLB of a CPU
PICA	purge from instruction cache of a CPU
PITLB	purge from instruction TLB of a CPU
REDO	edit and do a command from the command history stack
RESET	reset a Twin64 system or CPU
RUN	run a configured simulation
STEP	step through a simulation

Continued on next page

Command	Purpose
W	evaluate and display an expression
XF	execute commands from a file

Command Syntax

Command syntax is described using the following conventions. Reserved words are written in uppercase letters. Parameters begin with a lowercase letters

Optional elements are enclosed in square brackets `[ ]`. Elements that may be repeated are enclosed in curly braces { }.

Numeric values are shown with a `0x` prefix when expressed in hexadecimal notation. For improved readability, underscore characters may be used as digit separators within hexadecimal values.

String literals are enclosed in apostrophes (`' '`).

The next pages present each command in more detail, showing their purpose, syntax, arguments and an example.

DA - Display Physical Memory

Syntax

`da <adr> [ , <len> ] [ , <fmt> ]`

Description

The `DA` command displays data from physical memory, starting at address `adr` and continuing for `len` bytes. If `len` is omitted, a default length of 8 bytes is used.

For numeric data display, the starting address is rounded down to an 8-byte boundary. For code display, the starting address is rounded down to a 4-byte boundary.

The optional `fmt` parameter selects the output format. Supported formats include hexadecimal (default), decimal, and disassembled instruction words.

Example

The following example displays 16 bytes starting at address `0x1000`.

```
(3) ->da 0x1000, 10
00_0000_1000: 0000_0000_0000_0000  0000_0000_0000_0000

(4) ->
```

The `fmt` option can be used to display the data as disassembled instruction words.

```
(4) ->da 0x1000, 10, code
00_0000_4000: NOP
00_0000_4010: NOP
00_0000_4020: NOP

(5) ->
```

Notes

None.

DM - List Module Info

Syntax

```
dm [ <modNum> ]
```

Description

The `DM` command displays the configured modules of the Town64 system. Each module is listed with its module number, type, hard physical address (`HPA`), and the optional soft physical address range (`SPA`) and size.

If no module number is specified, all modules are displayed. If a module number is provided, only the selected module is shown.

Example

```
(1) ->dm
 Mod  Type   HPA              SPA              Size
 00   MEM    0x00_ffe0_0000   0x00_f000_0000   0000_4000
 01   MEM    0x00_ffe0_1000   0x00_0000_0000   0004_0000
 02   MEM    0x00_ffe0_2000   0x00_0004_0000   0004_0000
 03   PROC   0x00_ffe0_3000

(2) ->dm 2
 02   MEM    0x00_ffe0_2000   0x00_0004_0000   0004_0000

(3) ->
```

Notes

None.

DO - perform command from history stack

Syntax

```
do [ <cmdNum> ]
```

Description

The `DO` command re-executes a command from the command history stack. The optional command number `cmdNum` selects the entry to be executed.

If no command number is specified, the most recently entered command is re-executed. Command numbers correspond to the indices shown by the `HIST` command.

The `DO` command itself is not aded to the command history stack.

Example

The following example first displays the command history and then re-executes the command with history index `1`.

```
(4) ->hist
[0]: help
[1]: da 0x1000, 16
[2]: help
[3]: da 0x2000, 16
(4) ->do 1
00_0000_1000: 0000_0000_0000_0000  0000_0000_0000_0000
(5) ->
```

Notes

See also the `HIST` command for further information on the command history mechanism.

DW List Window Info

Syntax

```
dw [ <wNum> ]
```

Description

The DW command displays a list of all currently created windows. Each window is shown with its name and type, window ID, assigned stack number, andif applicablethe associated module number and module type.

Windows are listed regardless of whether they are enabled or disabled. If a window number is specified, only the selected window is displayed.

Specifying an optional stack number restricts the output to windows belonging to that stack.

Example

The following example shows windows created using the WN command. In this example, the module number for the CPU and TLB is 3, and the memory window starts at address 0x1000.

```
(5)  -> wn cpu, 3
(6)  -> wn ITLB, 3
(7)  -> wn MEM, 0x1000
(8)  -> dw
Name      Stack     Id        WType     Mod       MType
CPU       1         1         CPU       3         PROC
I-TLB     1         2         TLB       3         PROC
MEM       2         3         Memory    1         MEM
(9)  ->
(9)  -> dw 2
MEM       2         3         Memory    1         MEM
(10) ->
```

Notes

See also the WN command.

ENV Manage Environment Variables

Syntax

```
env [ <var> [ <val> ]] | env <var>
```

Description

The ENV command manages environment variables. An environment variable is a simple namevalue pair. Variable names must be unique and are case-insensitive. Names may contain underscores, but the first character must be alphabetic.

Invoking ENV with no arguments lists all defined variables. Providing only a variable name displays its value, if the variable exists. Providing both a variable name and a value sets the variable. If the variable does not exist, it is created. Providing a variable name followed by the - character removes the variable.

Example

```
(13) -> env
 TRUE                          BOOL:    TRUE
 FALSE                         BOOL:    FALSE
 PROG_VERSION                  STR:     "A.00.01"
 GIT_BRANCH                    STR:     "main"
 PATCH_LEVEL                   NUM:     1
 SHOW_CMD_CNT                  BOOL:    TRUE
 CMD_CNT                       NUM:     14
 ECHO_CMD_INPUT                BOOL:    FALSE
 EXIT_CODE                     NUM:     0
 RDX_DEFAULT                   NUM:     16
 WORDS_PER_LINE                NUM:     8
 WIN_MIN_ROWS                  NUM:     24
 WIN_TEXT_WIDTH                NUM:     90

(14) ->
(14) -> env A_VAR 100
(15) -> env A_VAR
 A_VAR                         NUM:     100
(15) ->
(16) -> env A_VAR -
(17) ->
(17) -> env A_VAR
 ENV variable not found
(18) ->
```

Notes

None.

EXIT Leave Simulator

Syntax

```
exit | e [ <val> ]
```

Description

The `EXIT` command terminates the simulator. An optional return value may be provided and is passed to the calling environment.

Both `EXIT` and its short form `E` are accepted as command aliases.

Example

```
(52) -> exit 100
```

Notes

None.

FDCA Flush Data Cache

Syntax

```
fdca <vAdr>
```

Description

The `FDCA` command flushes a single cache line from the data cache. The cache line is selected using the specified virtual address.

For processors with a unified instruction and data cache, the command refers to the unified cache.

This command requires the current window to be a data cache window.

Example

The following command flushes the data cache line corresponding to the virtual address `0_0001_ffff_0000`.

```
(10) -> fdca 0_0001_ffff_0000
(11) ->
```

Notes

None.

FICA Flush Instruction Cache

Syntax

```
fdca <vAdr>
```

Description

The `FICA` command flushes a single cache line from the instruction cache. The cache line is selected using the specified virtual address.

For processors with a unified instruction and data cache, the command refers to the unified cache.

This command requires the current window to be a data cache window.

Example

The following command flushes the data cache line corresponding to the virtual address `0_0001_ffff_0000`.

```
(10) -> fica 0_00010_0000_1000
(11) ->
```

Notes

None.

HIST List Command History

Syntax

`hist [ <depth> ]`

Description

The simulator command interpreter maintains a command history stack. Each commandexcept for `DO` and `REDO`is added to this stack.

The `HIST` command lists the contents of the history stack, starting with the oldest entry and ending with the most recent one.

If an optional depth is specified, only the most recent `<depth>` entries are displayed.

Example

The following example shows the command history stack. The `HIST` command itself is not added to the stack.

```
(11) -> hist
[0]: env A_VAR 100
[1]: env
[2]: env A_VAR
[3]: help fdca
[4]: wn dcache, 3
[5]: wn icache, 3
[6]: fdca
[7]: fdca 0
[8]: wn cpu, 3
[9]: fdca 1
[10]: fdca 0
(11) ->
```

Notes

None.

IDTLB - insert into data TLB

Syntax `IDTLB`

Description The `IDTLB` command ...

Example

...

Notes None.

IITLB - insert into instruction TLB

Syntax `IITLB`

Description The `IITLB` command ...

Example

...

Notes None.

LF - load ELF file

Syntax `LF`

Description The `LF` command ...

Example

...

Notes None.

MA - modify physical memory

Syntax `MA`

Description The `MA` command ...

Example

...

Notes None.

MR - modify register

Syntax `MR`

Description The `MR` command ...

Example

...

Notes None.

PDCA - purge data cache

Syntax `PDCA`

Description The `PDCA` command ...

Example

...

Notes None.

PDTLB - purge data TLB

Syntax `PDTLB`

Description The `PDTLB` command ...

Example

...

Notes None.

PICA - purge from instruction cache

Syntax `PICA`

Description The `PICA` command ...

Example

...

Notes None.

PITLB - purge from instruction TLB

Syntax `PITLB`

Description The `PITLB` command ...

Example

...

Notes None.

REDO - redo command from history stack

Syntax `REDO`

Description The `REDO` command ...

Example

...

Notes None.

RESET - reset simulator

Syntax `RESET`

Description The `RESET` command ...

Example

...

Notes None.

RUN - run simulation

Syntax `RUN`

Description The `RUN` command ...

Example

...

Notes None.

STEP - step in a simulation

Syntax `STEP`

Description The `STEP` command ...

Example

...

Notes None.

W - display an expression value

Syntax `W`

Description The `W` command ...

Example

...

Notes None.

XF - execute commands from a file

Syntax `XF`

Description The `XF` command ...

Example

...

Notes None.

Simulator Window Commands

Window commands apply to windows only. They feature window creation, arrangement and navigation.

Command Syntax

Table 13: Simulator Window Commands

Command	Purpose
CWC	clear command window
CWC	set command window lines
WB	scroll window backward
WC	set window current
WD	disable window
WDEF	set window defaults
WE	enable window
WF	scroll window forward
WH	jump to window home index
WJ	jump to window index
WK	delete window
WL	set window lines
WN	create window
WOFF	disable all windows
WON	enable all windows
WR	set window radix
WS	assign window to stack
WSD	disable window stack
WSE	enable window stack
WT	toggle window display
WX	exchange window

Command Syntax

Command syntax is described using the following conventions. Reserved words are written in uppercase letters. Parameters begin with a lowercase letters

Optional elements are enclosed in square brackets `[ ]`. Elements that may be repeated are enclosed in curly braces { }.

Numeric values are shown with a `0x` prefix when expressed in hexadecimal notation. For improved readability, underscore characters may be used as digit separators within hexadecimal values.

String literals are enclosed in apostrophes (`' '`).

The next pages present each window command in more detail, showing their purpose, syntax, arguments and an example.

CWC - clear command window

Syntax `CWC`

Description The `CWC` command ...

Example

...

Notes None.

CWL - set command window lines

Syntax `CWL`

Description The `CWL` command ...

Example

...

Notes None.

WB - scroll window backward

Syntax `WB`

Description The `WB` command ...

Example

...

Notes None.

WC - set current window

Syntax `WC`

Description The `WC` command ...

Example

...

Notes None.

WD - disable window

Syntax `WD`

Description The `WD` command ...

Example

...

Notes None.

WDEF - set to window defaults

Syntax `WDEF`

Description The `WDEF` command ...

Example

...

Notes None.

WE - enable window

Syntax `WE`

Description The `WE` command ...

Example

...

Notes None.

WF - scroll window forward

Syntax `WF`

Description The `WF` command ...

Example

...

Notes None.

WH - set window to home position

Syntax `WH`

Description The `WH` command ...

Example

...

Notes None.

WJ - window jump

Syntax `WJ`

Description The `WJ` command ...

Example

...

Notes None.

WK - delete a window

Syntax `WK`

Description The `WK` command ...

Example

...

Notes None.

WL - set window lines

Syntax `WL`

Description The `WL` command ...

Example

...

Notes None.

WN - create a window

Syntax `WN`

Description The `WN` command ...

Example

...

Notes None.

WOFF - turn window display off

Syntax `WOFF`

Description The `WOFF` command ...

Example

...

Notes None.

WON - turn window display on

Syntax `WON`

Description The `WON` command ...

Example

...

Notes None.

WR - set window radix

Syntax `WR`

Description The `WR` command ...

Example

...

Notes None.

WS - set window to stack

Syntax `WS`

Description The `WS` command ...

Example

...

Notes None.

WSD - disable window stack

Syntax `WSD`

Description The `WSD` command ...

Example

...

Notes None.

WSE - enable window stack

Syntax `WSE`

Description The `WSE` command ...

Example

...

Notes None.

WT - toggle window content

Syntax `WT`

Description The `WT` command ...

Example

...

Notes None.

WX - exchange window ID

Syntax `WX`

Description The `WX` command ...

Example

...

Notes None.

Simulator Predefined Functions

ASM - ...

Syntax `asm ( argStr )`

Description The `ASM` predefined function ...

Example

...

Notes None.

DISASM - ...

Syntax `disasm ( instr )`

Description The `DISASM` predefined function ...

Example

...

Notes None.

Simulator Variables

Simulator Configuration

Part VII

Processor Design

Processor Design

Pipeline

Twin-64 has a three stage pipeline, with each stage have two subStages.

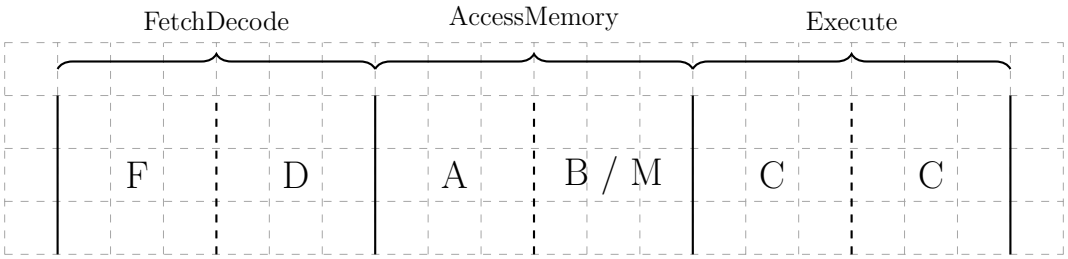


Figure 38: Single Instruction Pipeline

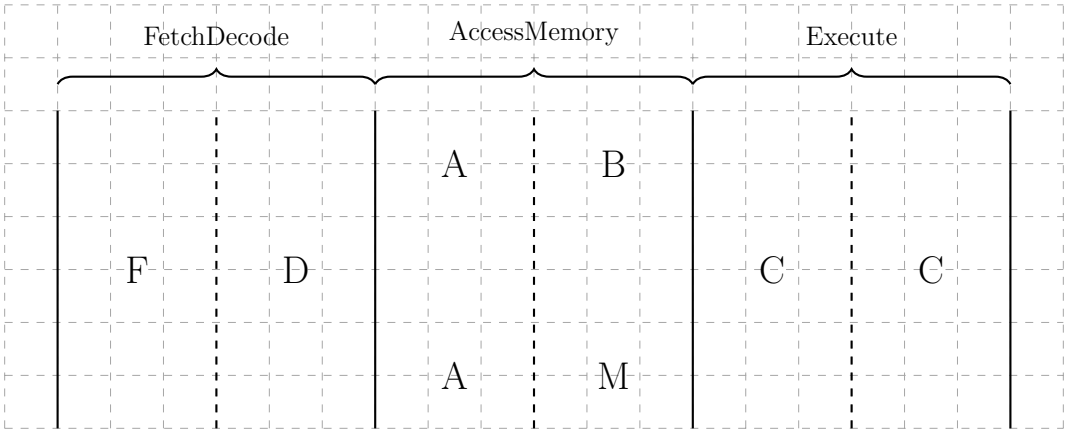


Figure 39: Dual Instruction Pipeline

Two instructions take pace in parallel.

Instructions are Compute, Branch, Memory, System.

Combinations possible are: CM, CB, CS, CC, BM.

Address computation uses 32-bit adders.

ALU is 64 Bit. We only need one, since we can compute two substages one result each.

Part VIII

Appendix

Appendix - Instruction Commentary

This appendix presents extended commentary on selected instruction groups. It provides further explanation, design rationale, and hardware-level insights that complement the main instruction descriptions. Each subsection discusses the motivation, encoding, and implementation strategy for one or more related instructions, offering a deeper view into the processors architectural design.

Arithmetic Operation Instructions

ADD, SUB, SHLxA, SHRxA

Bit Operation Instructions

Boolean operations

AND, OR, XOR have a C and N flag.

Bit operations with immediate operand

For bit operation instructions that use an immediate value as one operand, there are two basic design options for interpreting the immediate field: it may be treated as an unsigned value and zero-extended to the machine word size, or it may be interpreted as a signed value and sign-extended. A zero-extended immediate is convenient for constructing small low-bit masks, while a sign-extended immediate provides access to negative constants, which correspond to masks with many high bits set.

This architecture adopts *sign-extension* for logical immediate values. The primary motivation is uniformity: the logical I-type instructions share the same immediate decoding path as arithmetic instructions such as `ADDI`, avoiding additional decode logic.

Sign-extension also enables several useful bit-manipulation idioms without requiring extra instructions. For example, `AND Rn, Rm, -2` uses the sign-extended mask `0x...FFFE` to clear the least significant bit, and `AND Rn, Rm, -4096` produces the mask `0x...F800`, which aligns an address to a 4 KiB boundary.

Similarly, `XOR Rn, Rm, -1` yields the all-ones mask `0x...FFFF`, providing a compact encoding of bitwise NOT without a dedicated instruction. The instruction `OR Rn, Rm, -256` constructs the high-bit mask `0x...FF00`, useful when forcing upper bits to one.

Using sign-extension for logical immediate values therefore offers both a straightforward hardware implementation and a flexible set of common masking and bit-twiddling operations.

Bit operations Extract, Deposit and Shift

often used: mask and ANDm which we do with EXTR for example...

Bit operations Shift and Rotate

EXTR, DEP, DSR

Twin-64 does not offer instructions to perform rotate and shift instructions. They are implemented using the `EXTR`, `DEP` and `DSR` instructions.

Comparison and Branch Condition Encoding

This appendix explains the condition encoding and implementation philosophy used by the `CMP`, `CBR`, `ABR`, and `MBR` instructions. The design goal is to minimize hardware complexity by reusing a single subtraction/comparator path. Comparison operators are encoded in a compact and symmetric way that makes condition inversion trivial.

The following instructions use the same 3-bit comparison field (`cond`):

- **CMP** Compares two operands using the 3-bit `cond` field and produces a boolean result (0/1) in a register.
- **CBR** Compares two registers and branches based on the same 3-bit `cond` field. This performs a direct comparison without requiring a preceding `CMP`.
- **ABR** Performs an add operation and branches using a 3-bit `cond` field that tests the *add result* against zero or other simple predicates. Intended for loop counters or pointer walking.
- **MBR** Performs a move operation and branches using the same 3-bit `cond` field. The comparison is applied to the source operand, typically against zero.

All four instructions share a uniform 3-bit condition encoding, allowing common decoder and comparator logic. Branch target computation uses a sign-extended 15-bit offset shifted left by two, allowing word-aligned branch destinations within a `64 KB` range relative to the current instruction address.

Design Rationale The condition encoding is designed for symmetry and minimal logic cost. Conditions are arranged in complementary pairs so that flipping a single bit yields the inverse relation. This symmetry makes branch inversion trivial in hardware and simplifies assembler implementation.

Value	Binary	Mnemonic	Meaning / Branch if
0	000	`EQ`	Equal (`==`)
1	001	`LT`	Less than (`<`)
2	010	`GT`	Greater than (`>`)
3	011	`EV`	Even (bit 0 = 0)
4	100	`NE`	Not equal (`!=`)
5	101	`GE`	Greater than or equal (`>=`)
6	110	`LE`	Less than or equal (`<=`)
7	111	`OD`	Odd (bit 0 = 1)

The most significant bit (`cond[2]`) acts as an *invert bit*: flipping it inverts the sense of the comparison. The relationships between base and inverted conditions are as follows:

Base Condition	Inverted Condition	Encoding Pair
`EQ`	`NE`	000 / 100
`LT`	`GE`	001 / 101
`GT`	`LE`	010 / 110
`EV`	`OD`	011 / 111

This arrangement keeps decoding logic compact and uniform across all instruction forms.

Logical Decoding and Hardware Signals All conditions used by the comparator and branch unit are derived from simple signals computed by the ALU after a subtraction, or directly from the result when testing against zero. The example below illustrates a possible Verilog implementation:

```
wire Z  = (result == 64'd0);  // Zero flag
wire S  = result[63];         // Sign flag (for signed comparisons)
wire B0 = result[0];          // Least significant bit (for EV/OD)

wire cond_EQ =  Z;
wire cond_LT =  S;
wire cond_GT = (~S) & (~Z);
wire cond_EV = ~B0;

wire cond_NE = ~Z;
wire cond_GE = ~S;
wire cond_LE =  S | Z;
wire cond_OD =  B0;

assign branch_taken =
    (cond == 3'b000) ? cond_EQ :
    (cond == 3'b001) ? cond_LT :
    (cond == 3'b010) ? cond_GT :
    (cond == 3'b011) ? cond_EV :
    (cond == 3'b100) ? cond_NE :
    (cond == 3'b101) ? cond_GE :
    (cond == 3'b110) ? cond_LE :
                       cond_OD;
```

All comparisons are signed by default; unsigned comparisons can be implemented by treating operands as unsigned integers or through a future `CMPU` variant if required. The `EV`/`OD` conditions allow efficient branching on bit 0, which is useful for address alignment tests, distinguishing even/odd indices, and loop unrolling.

Because all comparison and branch instructions share this common encoding and logic, the branch unit requires only a single multiplexer and no dedicated condition register, simplifying both the datapath and verification.

Control Flow Instructions

BB, B, BR, BV, BE

CBR, ABR, MBR

Immediate Instructions

ADDIL, LDI, LDO

Memory Reference Instructions

Twin-64 provides conventional load and store instructions together with a small, carefully chosen set of addressing modes. In addition, the computational instructions `ADD`, `SUB`, `AND`, `OR`, `XOR`, and `CMP` may use these addressing modes to obtain their second operand. This reflects the register-memory character of the architecture.

Addressing Modes and Data Access Model

Twin-64 supports two fundamental and orthogonal addressing modes for data access:

1. Base register plus immediate offset, and
2. Base register plus index register.

These modes form the basis of all load/store operations and all computational instructions that reference memory for their second operand.

Scaled Indexing and Implicit Alignment When the index-register mode is used, the index value is automatically scaled according to the size of the referenced data item. For example, a half-word access shifts the index left by one bit, while a word access shifts it left by two bits. This scaling increases the effective range of the index and ensures that the resulting address is aligned to the natural boundary of the accessed data. The effective address is computed as the sum of the base register and the scaled index.

Signed Data Semantics All data fetched by load instructions or by memory-operand computational instructions is interpreted as a signed quantity. Bytes, half-words, and words are sign-extended to the full 64-bit register width during access. This design aligns with the Twin-64 arithmetic model, simplifies the ALU and trap semantics, and avoids the complexity of maintaining separate signed and unsigned data paths.

Absence of Pre/Post-Increment Addressing Some historical architectures provided pre-increment and post-increment addressing modes, in which the effective address is both computed and written back to the base register. Twin-64 intentionally omits these modes.

Implicit updates require additional register-file write ports or multi-cycle update paths, increasing datapath and hazard complexity. They also introduce subtle ordering and forwarding constraints that complicate pipeline timing. Since modern compilers seldom use such addressing forms, their practical value does not justify the associated hardware and verification cost. Their omission helps preserve the simplicity and implementation efficiency of Twin-64.

Load-Reserved and Store-Conditional Instructions

The `LDR` (Load Reserved) and `STC` (Store Conditional) instructions provide the architectural mechanism for implementing atomic read-modify-write sequences without requiring explicit locks. Together, they form the foundation for synchronization primitives such as semaphores, spinlocks, and atomic counters.

- **LDR** loads a 64-bit value from memory and establishes a *reservation* on the cache line containing that address. The effective address is computed as the sum of the base register and the signed offset and must be 8-byte aligned.
- **STC** attempts to store the value in `Rn` to the same memory location, succeeding only if the reservation remains valid. If the reservation is valid, `STC` completes normally and returns a success indication (typically a value of 1). If the reservation has been lost, the store is suppressed and a failure value (0) is returned to the destination register.

Design Rationale The `LDR`/`STC` pair allows atomic sequences to be expressed entirely in software, avoiding pipeline stalls and bus locks. The mechanism relies on a single internal reservation register that tracks the address (or cache line) of the most recent `LDR`. Only one reservation may be active per processor at any time. Using the cache system as the underlying reservation tracker reduces hardware cost and ensures natural invalidation under normal cache-coherency events. A reservation is cleared automatically under any of the following conditions:

- The cache line containing the reserved address is invalidated or evicted through the local or other processors.
- The local processor performs a write to the same cache line through any path other than `STC`.
- An interrupt, trap, or context switch occurs.
- A subsequent `LDR` instruction establishes a new reservation.

This model guarantees that at most one `STC` can succeed per `LDR`, ensuring forward progress while maintaining strong atomicity at the cache-line granularity.

Implementation Notes The reservation register may store the cache line tag rather than the full address, allowing fast comparison against cache operations. Integration with the cache controller ensures that any coherence or invalidation event automatically clears the reservation flag. Because interrupts also clear reservations, interrupt latency and atomicity properties remain predictable.

This design matches the behavior of similar primitives in other architectures (e.g., RISC-V `LR/SC` or PowerPC `lwarx/stwcx.`) but uses a cache-linebased validity rule rather than a fixed address match, simplifying coherence handling in multiprocessor systems.

Atomic Increment Example The following example implements an atomic increment function. The memory location is specified by base register GR5 and an offset.

```
atomic_inc: LDR     R1, ofs( R5 )         ; Load current value and set reservation
            ADD     R1, R1, 1             ; Increment locally
            STC     R1, ofs( R5 )         ; Attempt conditional store
            CBR.EQ  R1, R0, atomic_inc    ; Retry if store failed (R1 = 0)
            BE (RL)                       ; return
```

Each iteration performs a reserved load, modifies the value, and attempts a conditional store. If another processor writes to the same cache line between the `LDR` and `STC`, the reservation is cleared, causing `STC` to fail and the loop to retry. This ensures that the final store reflects an atomic update to memory.

Compare-and-Swap (CAS) Example The following example implements the compare and swap function.

```
;   Arguments:
;      Arg0   : base address of the value location
;      Arg1   : expected old value
;      Arg2   : desired new value
;
;   Returns:
;      Arg0  : 1 on success, 0 on failure

cas:          LDR    R1, 0(Arg0)           ; load reserved value
              CBR.EQ R1, Arg1, cas_fail    ; compare with expected value
                                           ; If not equal, fail
              STC    Arg2, 0(Arg0)         ; Attempt conditional store
              MBR    Arg2, Arg2, cas       ; Retry if store failed
              OR     Arg0, R0, 1           ; Indicate success
              BE ( RL )                    ; return

cas_fail:     OR     Arg0, R0, 0           ; Indicate failure
              BE ( RL )                    ; return
```

This CAS sequence allows software-based atomic updates: the memory location is only updated if its current value matches the expected value. Failed stores due to lost reservations cause the loop to retry, guaranteeing atomicity even in multiprocessor environments.

Summary Together, these examples demonstrate how the `LDR`/`STC` mechanism enables fully software-based atomic readmodifywrite operations. By combining reserved loads with conditional stores, the processor can implement synchronization primitives, spinlocks, and lock-free data structures without additional hardware beyond the reservation register and cache-line tracking.

Appendix - Instruction Notation Routines

describe the routines used in the instruction set operation part...

Table 14: Instruction Notation Routines

Function	Description
addOfs32(base, ofs)	the offset is added to the base register using a 32-but unsigned arithmetic. The upper portion of the base register remains unchanged.
immOperand(instr)	the singed 15-bit immediate value field in the instruction is selected and sign extended.
adrImmIndexed(instr)	the signed 13-bit immediate value field in the instruction is selected, shifted left by the dw field and sign extended. It is added to the base register RegB using 32-bit unsigned arithmetic. The upper 32-bits of RegB remain unchanged.
adrRegIndexed(instr)	the lower 32-bits of RegX are shifted left by the dw field and sign extended. The value is added to the base register RegB using 32-bit unsigned arithmetic. The data is fetched according to the data width.
memLoadImmIndexed(instr)	the signed 13-bit immediate value field in the instruction is selected, shifted left by the dw field and sign extended. It is added to the base register RegB using 32-bit unsigned arithmetic. The upper 32-bits of RegB remain unchanged.
memLoadRegIndexed(instr)	the lower 32-bits of RegX are shifted left by the dw field and sign extended. The value is added to the base register RegB using 32-bit unsigned arithmetic. The data is fetched according to the data width.
memStoreImmIndexed(instr, data)	the signed 13-bit immediate value field in the instruction is selected, shifted left by the dw field and sign extended. It is added to the base register RegB using a 32-bit unsigned arithmetic. The upper 32 bits of RegB remain unchanged. The data is stored according to the data width.

Continued on next page

Function	Description
memStoreImmIndexed(instr, data)	the lower 32-bits of RegX are shifted left by the dw field and sign extended. The value is added to the base register RegB using a 32-bit unsigned arithmetic. The upper 32 bits of RegB remain unchanged. The data is stored according to the data width.
testBit(reg, pos)	the bit in the passed register at the position is tested.
compare(cond, reg1, reg2)	Reg2 is subtracted from Reg1 and evaluated for the condition. If the condition is true, a 1 is returned.
deposit(reg1, reg2, pos, len)	The right most field of len in Reg 2 is extracted and deposited at the position in Reg 1.
extract(reg1, reg2, pos, len)	The field of len at the position in Reg2 is extracted and returned in Reg1.
doubleShiftR(reg1, reg2, len)	Reg1 and Reg2 are concatenated and shifted right by len. The rightmost 64 bits of the result are returned.
diagOperation(id, reg1, reg2)	Diagnostic function id is passed reg1 and reg2.
flushFromCache(adr)	Cache flush operation.
purgeFromCache(adr)	Cache purge operation.
insertIntoTlb(adr, info)	TLB insert.
purgeFromTlb(adr, info)	TLB purge.

Appendix - Programming Notes

??? to be defined....

Appendix -Diagnostic Functions

Table 15: Diagnostic Functions

Id	Function	Arg0	Arg1
0	No operation	0	0
1	Write status display	value	-

Appendix - Glossary

Absolute Address An address in the physical memory range from zero to the maximum physical memory size. This range is directly mapped to an equivalent virtual address range.

Address Space A logical domain of addresses. The CPU may define multiple address spaces, such as *main memory space* and *I/O space*. Each space defines an independent address range and access semantics.

Callee Save Registers Registers whose values must be preserved across a function call by the callee (the function being called). If the callee uses any of these registers, it must save their original values on entry (typically on the stack) and restore them before returning. This convention allows callers to assume the registers retain their values after the call.

Caller Save Registers Registers that may be overwritten by a called function. The caller must save these registers (if it needs their values later) before invoking another function. This convention allows the callee to use these registers freely without preserving their previous contents.

ELF Section A named subdivision in an ELF file used by the linker and compiler (e.g., `.text`, `.data`, `.bss`). Sections are grouped by the linker into ELF segments.

ELF Segment A loadable portion of an ELF file (defined by a program header). Each ELF segment is mapped into one or more regions of an address space during program loading.

Function Pointer A reference to a function descriptor.

Global Data Area All modules compiled for a load module allocate their global data in this area. The DP register points to the beginning of the area.

Global Data Pointer (DP) The address of the global data area. The address is normally kept in the DP register. Each module has a DP area in the global data area. Load modules also use this register to access the linkage table.

IA-relative Addressing Code that uses the current instruction address (IA) as the base for branches and for accessing constant data.

Linkage Table A table of address descriptors for functions external to the load module. One type of entry stores the address of an external function. Another type stores a reference to the external load module's global data area.

Load Module An executable unit of code produced by the assembler or compiler from one or several modules.

Mapped Region A runtime mapping that associates an ELF segment or memory source with a region in an address space. The emulator or loader tracks mapped regions to manage memory access and protection.

Object A unit of storage within a region. Objects correspond to program-visible entities or runtime allocations, such as global variables, heap blocks, or stack frames. Examples: *global object*, *heap object*, *stack object*.

Offset An unsigned 32-bit value that, together with the region, forms a virtual address. Examples: *code offset*, *data offset*.

Region A contiguous range of addresses within the virtual address space, reserved for a particular purpose. A region represents the upper 20 bits of the 52-bit virtual address range. Examples: *code region*, *data region*, *stack region*.

Virtual Address A virtual address consists of the region ID and the region offset. Together they form the architected virtual address space of 52 bits.

Appendix -Notes

just notes to work into the document:

TLB

Twin-64 TLBs support variable page sizes, aligned to the page size. The hash function still hashes to the base page when traversing the page table. The page table is now a set of entries rather than a one-to-one mapping between physical and virtual pages. Consequently, we maintain a table with one entry per physical page for memory management.

On a TLB miss, we start with the base page index and walk the collision chain. If no entry is found, we then examine the next larger region size, hashing the start address of that region. If the region is not found in any chain, a page fault is raised.

The main challenge arises during write-back. We want to write back only the modified portions, not the entire region. To achieve this, a region is initially allocated as read-only. Any write attempt triggers a trap, at which point the region is split into two parts: one remains read-only, and the other is marked dirty. The split size is critical, as the size of the dirty portion cannot be changed later.

In practice, large regions are typically read-only and contain data such as libraries or program code. As a result, write-back situations are relatively rare, and the overhead of the write-back algorithm is justified.